OPERATING SYSTEMS NOTES

Theory explained step by step

Author: SamuMGH04

WELCOME

Thanks for choosing these notes. This project is created with the aim of making people understand what's behind an Operating System and everything related to it.

LICENSE

These works are protected and distributed under the license
Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International

- **Attribution:** You must mention the original creator
- **Non Commercial:** You can't sell this material
- **No Derivatives:** You can't modify or change the text.

SISTEMI OPERATIVI: LEZIONE 5

Contents

1 Come Funziona la Memoria nei Sistemi Moderni **8**
1.1 La memoria come array di byte . . . 8
1.2 Il ruolo del Program Counter . . . 8
1.3 Istruzioni e operazioni di memoria . . . 9
1.4 Il ciclo di esecuzione delle istruzioni . . . 9
1.5 La memoria come osservatore passivo . . . 9
1.6 Il flusso di indirizzi come oggetto di analisi . . . 10

2 CPU, Memoria e il Ruolo della Cache **11**
2.1 Aree di memoria direttamente accessibili dalla CPU . . . 11
2.2 Localizzazione delle istruzioni e dei dati . . . 11
2.3 Velocità dei registri della CPU . . . 11
2.4 Lentezza relativa della memoria principale . . . 12
2.5 Stalli della CPU . . . 12
2.6 Introduzione della cache . . . 12
2.7 Benefici della cache . . . 13

3 Protezione della Memoria tramite Registri di Base e Limite **13**
3.1 La protezione della memoria come requisito di sicurezza . . . 13
3.2 Perché la protezione deve essere implementata in hardware . . . 13
3.3 Isolamento dei processi . . . 14
3.4 Definizione dell'intervallo di indirizzi legali . . . 14
3.5 Registri hardware per la protezione: Base e Limite . . . 14
3.6 Funzionamento del controllo di protezione . . . 15
3.7 Importanza del meccanismo Base-Limit . . . 15

4 Protezione della Memoria (Registri di Base e Limite) **16**
4.1 Controllo degli indirizzi in modalità utente . . . 16
4.2 Gestione degli accessi illegali: trap e terminazione del processo . . . 16
4.3 Privilegi del sistema operativo: modifica dei registri di base e limite . . . 17
4.4 Accesso completo alla memoria da parte del kernel . . . 17
4.5 Sintesi del meccanismo di protezione . . . 17

5 Caricamento di un Programma in Memoria **18**
5.1 Il programma come file binario su disco . . . 18
5.2 Dal file al processo: creazione del contesto di esecuzione . . . 18
5.3 Accesso alla memoria durante l'esecuzione . . . 18
5.4 Terminazione del processo e rilascio della memoria . . . 19

5.5 Flessibilità nel posizionamento in memoria 19
5.6 Vantaggi della flessibilità nel caricamento 19

6 Quando Avviene il Binding degli Indirizzi 20
6.1 Evoluzione degli indirizzi: da simbolici ad assoluti 20
6.2 Binding durante la compilazione 20
6.3 Binding durante il caricamento 20
6.4 Binding durante l'esecuzione 21
6.5 Motivazioni per il binding a run-time nei sistemi moderni 21

7 Indirizzi Logici e Indirizzi Fisici 22
7.1 Indirizzi logici generati dalla CPU 22
7.2 Indirizzi fisici utilizzati dall'hardware 22
7.3 Corrispondenza tra indirizzi logici e fisici 22
7.4 Binding a run-time: indirizzi logici diversi da quelli fisici 23
7.5 Spazio degli indirizzi logici 23
7.6 Spazio degli indirizzi fisici 23
7.7 Importanza della distinzione 24

8 MMU e Rilocazione degli Indirizzi 24
8.1 Funzione della Memory Management Unit 24
8.2 MMU semplice basata su registro di rilocazione 24
8.3 Esempio di traduzione tramite registro di rilocazione 25
8.4 Il programma vede solo indirizzi logici 25
8.5 Traduzione trasparente degli indirizzi 25

9 Caricamento Dinamico 26
9.1 Limiti dell'esecuzione tradizionale 26
9.2 Principio del caricamento dinamico 26
9.3 Formato relocatable load 26
9.4 Esecuzione con caricamento dinamico 27
9.5 Vantaggi del caricamento dinamico 27

10 Linking Dinamico e Librerie DLL 28
10.1 Librerie collegate dinamicamente 28
10.2 Vantaggi rispetto al linking statico 28
10.3 Funzionamento del linking dinamico 29
10.4 Aggiornabilità indipendente delle DLL 29
10.5 Curiosità: i file `.dll` nei sistemi Windows 30

11 Allocazione Contigua della Memoria 31
11.1 Coesistenza di sistema operativo e processi utente 31
11.2 Divisione della memoria in due regioni principali 31
11.3 Presenza simultanea di più processi 31
11.4 Allocazione contigua: un processo, un blocco 32
11.5 Assegnazione dei blocchi ai processi 32

12 Protezione della Memoria con Registri di Rilocazione e Limite **33**
12.1 Protezione tramite combinazione di rilocazione e limite 33
12.2 Intervallo di indirizzi validi 33
12.3 Ruolo della MMU nella traduzione 33
12.4 Caricamento dei registri durante il context switch 34
12.5 Prevenzione degli accessi illegali 34
12.6 Flessibilità per il sistema operativo 34
12.7 Sintesi del meccanismo 34

13 Allocazione a Partizioni di Dimensione Variabile **35**
13.1 Partizioni di dimensione variabile 35
13.2 Tabella di gestione della memoria 35
13.3 Stato iniziale della memoria 35
13.4 Creazione di buchi durante la terminazione dei processi 36
13.5 Caricamento di nuovi processi nei buchi disponibili 36
13.6 Frammentazione della memoria 36

14 Gestione delle Richieste di Memoria e dei Buchi Liberi **37**
14.1 Valutazione delle richieste di memoria 37
14.2 Presenza di buchi di memoria di dimensioni variabili 37
14.3 Ricerca di un buco adatto 37
14.4 Rilascio della memoria alla terminazione del processo 38
14.5 Sintesi del meccanismo di gestione 38

15 Strategie di Allocazione Dinamica della Memoria **39**
15.1 First-fit 39
15.2 Best-fit 39
15.3 Worst-fit 39
15.4 Confronto tra le strategie 40

16 Frammentazione Esterna **40**
16.1 Cause della frammentazione esterna 40
16.2 Effetti della frammentazione esterna 40
16.3 La regola del 50 percento 40

17 Frammentazione Interna ed Esterna **41**
17.1 Frammentazione esterna 41
17.2 Frammentazione interna 41
17.3 Gestione dei piccoli frammenti 41
17.4 Compattazione 41
17.5 Soluzione più generale 42

18 Introduzione alla Paginazione **42**
18.1 Motivazioni per l'uso della paginazione 42
18.2 Divisione della memoria in blocchi di dimensione fissa 42
18.3 Vantaggi della dimensione fissa 43
18.4 Adozione nei sistemi moderni 43

19 Paginazione: Meccanismo di Base **44**
19.1 Divisione della memoria in frame e pagine 44
19.2 Caricamento delle pagine in memoria 44
19.3 Memoria secondaria e dimensione dei blocchi 44
19.4 Struttura dell'indirizzo logico 44
19.5 Tabella delle pagine 45
19.6 Traduzione dell'indirizzo 45
19.7 Sintesi del meccanismo 45

20 Traduzione degli Indirizzi nella MMU **46**
20.1 Generazione dell'indirizzo logico 46
20.2 Passaggi della traduzione eseguiti dalla MMU 46
20.3 Calcolo dell'indirizzo fisico 46
20.4 Sintesi del processo 47

21 Dimensione della Pagina e Struttura dell'Indirizzo Logico **47**
21.1 Dimensione della pagina 47
21.2 Struttura dell'indirizzo logico 48
21.3 Interpretazione dei campi 48
21.4 Vantaggi della struttura a potenze di 2 48

22 Traduzione degli Indirizzi: Esempio Numerico **49**
22.1 Parametri del sistema 49
22.2 Struttura dell'indirizzo logico 49
22.3 Rilocazione dinamica tramite paginazione 49
22.4 Passaggi della traduzione 50
22.5 Sintesi 50

23 Frammentazione Interna nella Paginazione **51**
23.1 Assenza di frammentazione esterna 51
23.2 Presenza di frammentazione interna 51
23.3 Esempio numerico 51
23.4 Limiti teorici della frammentazione interna 52
23.5 Trade-off nella scelta della dimensione della pagina 52
23.6 Dimensioni tipiche delle pagine nei sistemi moderni 52
23.7 Sintesi 52

24 Caricamento di un Processo in Memoria **53**
24.1 Requisito di frame liberi 53
24.2 Caricamento delle pagine nei frame 53
24.3 Vista del programmatore 53
24.4 Vista del sistema 53
24.5 Responsabilità del sistema operativo 54
24.6 Sintesi 54

25 Page Table e Context Switching **55**
25.1 Tabella delle pagine per processo 55
25.2 Operazioni durante il context switch 55
25.3 Memorizzazione delle tabelle delle pagine 55

25.4 Efficienza del cambio di tabella 56
25.5 Sintesi 56

26 Translation Lookaside Buffer (TLB) 57
26.1 Struttura e funzionamento della TLB 57
26.2 TLB hit e TLB miss 57
26.3 Politiche di rimpiazzo della TLB 58
26.4 Dimensione della TLB 58
26.5 Sintesi del ruolo della TLB 58

27 TLB e Address Space Identifiers (ASIDs) 59
27.1 ASID nella TLB 59
27.2 Vantaggi degli ASID 59
27.3 Assenza di ASID: necessità di flush 59
27.4 Sintesi 60

28 TLB Hit Ratio ed Effective Access Time (EAT) 60
28.1 Hit ratio 60
28.2 Tempi di accesso 60
28.3 Effective Access Time (EAT) 61
28.4 Importanza di un elevato hit ratio 61
28.5 Sintesi 61

29 Paginazione e Protezione della Memoria 62
29.1 Bit di protezione nella tabella delle pagine 62
29.2 Bit valid / invalid 62
29.3 Ruolo nella protezione della memoria 62
29.4 Sintesi 63

30 Valid–Invalid Bit: Esempio e Frammentazione Interna 63
30.1 Scenario dell'esempio 63
30.2 Valid–Invalid bit 63
30.3 Problema: frammentazione interna nell'ultima pagina 64
30.4 Perché questa area è accessibile? 64
30.5 Conseguenze 64
30.6 Sintesi 64

31 Paginazione e Codice Condiviso Reentrant 65
31.1 Codice reentrant 65
31.2 Condivisione delle pagine 65
31.3 Vantaggi della condivisione 65
31.4 Esempio numerico 66
31.5 Sintesi 66

32 Paginazione Gerarchica (Two-Level Paging) 67
32.1 Motivazione 67
32.2 Suddivisione dell'indirizzo logico 67
32.3 Struttura della paginazione a due livelli 67
32.4 Allocazione su richiesta 68

32.5 Vantaggi della paginazione gerarchica . . . 68
32.6 Sintesi . . . 68

33 Paginazione Gerarchica nei Sistemi a 64 Bit **69**
33.1 Dimensione dello spazio logico a 64 bit . . . 69
33.2 Limiti della paginazione a due livelli . . . 69
33.3 Aggiungere più livelli di paginazione . . . 69
33.4 Esempio: paginazione a 3 livelli . . . 70
33.5 Conclusione: la paginazione gerarchica non basta . . . 70
33.6 Sintesi . . . 70

34 Hashed Page Tables **71**
34.1 Motivazione . . . 71
34.2 Struttura della hashed page table . . . 71
34.3 Procedura di lookup . . . 71
34.4 Gestione delle collisioni . . . 72
34.5 Vantaggi delle hashed page tables . . . 72
34.6 Sintesi . . . 72

35 Inverted Page Tables **73**
35.1 Caratteristiche principali . . . 73
35.2 Contenuto di ogni entry . . . 73
35.3 Procedura di traduzione degli indirizzi . . . 73
35.4 Ricerca tramite hashing . . . 74
35.5 Vantaggi delle inverted page tables . . . 74
35.6 Limiti . . . 74
35.7 Sintesi . . . 74

36 Swapping **75**
36.1 Principio di funzionamento . . . 75
36.2 Vantaggi dello swapping . . . 75
36.3 Relazione con la multiprogrammazione . . . 75
36.4 Sintesi . . . 76

37 Standard Swapping **76**
37.1 Caratteristiche dello standard swapping . . . 76
37.2 Processi ideali per lo swap out . . . 76
37.3 Swap in . . . 77
37.4 Vantaggi dello standard swapping . . . 77
37.5 Limiti . . . 77
37.6 Sintesi . . . 77

38 Paged Swapping (Swapping Moderno) **78**
38.1 Caratteristiche del paged swapping . . . 78
38.2 Operazioni fondamentali . . . 78
38.3 Vantaggi del paged swapping . . . 78
38.4 Perché è preferito nei sistemi moderni . . . 79
38.5 Sintesi . . . 79

39 Memory Management nei Sistemi Operativi Mobile **80**
39.1 Perché i sistemi mobile non usano lo swapping . . . 80
39.2 iOS: gestione della memoria . . . 80
39.3 Android: gestione della memoria . . . 81
39.4 Conseguenze per gli sviluppatori . . . 81
39.5 Sintesi . . . 81

40 Esercizio 1: Bit dell'indirizzo logico e fisico **82**
40.1 Testo . . . 82
40.2 Passo 1: Interpretare i dati . . . 82
40.3 Passo 2: Calcolare i bit per il numero di pagina logica . . . 82
40.4 Passo 3: Calcolare i bit per l'offset . . . 82
40.5 Passo 4: Comporre l'indirizzo logico . . . 83
40.6 Passo 5: Calcolare i bit per il numero di frame fisico . . . 83
40.7 Passo 6: Comporre l'indirizzo fisico . . . 83
40.8 Riepilogo Esercizio 1 . . . 83

41 Esercizio 2: First-Fit, Best-Fit, Worst-Fit **84**
41.1 Testo . . . 84
41.2 Passo 1: Riscrivere i dati in forma strutturata . . . 84
41.3 First-Fit: passo per passo . . . 84
41.4 Best-Fit: passo per passo . . . 86
41.5 Worst-Fit: passo per passo . . . 88
41.6 Riepilogo finale Esercizio 2 . . . 90

42 Esercizio 3: Page Table Convenzionale vs Inverted Page Table **91**
42.1 Testo . . . 91
42.2 Passo 1: Interpretare i parametri . . . 91
42.3 Passo 2: Calcolare il numero di pagine logiche . . . 91
42.4 Passo 3: Page table convenzionale (single-level) . . . 92
42.5 Passo 4: Calcolare il numero di frame fisici . . . 92
42.6 Passo 5: Inverted page table . . . 92
42.7 Riepilogo Esercizio 3 . . . 93

43 Esercizio 4: Bit dell'indirizzo logico e fisico **93**
43.1 Testo . . . 93
43.2 Passo 1: Analisi dei dati . . . 93
43.3 Passo 2: Calcolare i bit per il numero di pagina logica . . . 93
43.4 Passo 3: Calcolare i bit per l'offset . . . 94
43.5 Passo 4: Comporre l'indirizzo logico . . . 94
43.6 Passo 5: Calcolare i bit per il numero di frame fisico . . . 94
43.7 Passo 6: Comporre l'indirizzo fisico . . . 94
43.8 Riepilogo Esercizio 4 . . . 94

1 Come Funziona la Memoria nei Sistemi Moderni

In questa sezione studiamo in modo rigoroso e approfondito il funzionamento della memoria nei sistemi di calcolo contemporanei. L'obiettivo è comprendere non solo la descrizione operativa, ma anche il significato concettuale di ogni elemento, il ruolo delle componenti architetturali e le implicazioni per l'analisi delle prestazioni.

1.1 La memoria come array di byte

La memoria principale viene modellata come un *grande array di byte*, ciascuno identificato da un *indirizzo univoco*. Formalmente possiamo rappresentare la memoria come una funzione:

$$\text{Memoria} : \{0, 1, \ldots, N-1\} \longrightarrow \{0, 1, \ldots, 255\},$$

dove:

- l'insieme $\{0, 1, \ldots, N-1\}$ rappresenta tutti gli indirizzi validi della memoria fisica;
- ogni indirizzo identifica un singolo byte;
- il valore associato a ciascun indirizzo è un numero intero tra 0 e 255, cioè il contenuto del byte.

Questa rappresentazione è fondamentale perché fornisce un modello astratto e uniforme: la memoria non distingue tra codice, dati, stack, heap o strutture complesse. Tutto è visto come una sequenza lineare di byte indirizzabili.

1.2 Il ruolo del Program Counter

La CPU esegue un programma recuperando le istruzioni dalla memoria. Questo processo è guidato dal *Program Counter* (PC), un registro che contiene l'indirizzo della prossima istruzione da eseguire.

Il PC svolge due funzioni essenziali:

1. indica alla CPU dove si trova l'istruzione corrente;
2. viene aggiornato automaticamente dopo ogni istruzione, salvo modifiche esplicite (come salti o chiamate di funzione).

Il fatto che il PC punti a un indirizzo di memoria rafforza l'idea che il codice sia trattato come dati: la CPU non fa distinzione semantica, interpreta semplicemente il contenuto dell'indirizzo come un'istruzione.

1.3 Istruzioni e operazioni di memoria

Molte istruzioni, oltre a essere esse stesse memorizzate in memoria, richiedono ulteriori operazioni di memoria durante la loro esecuzione. In particolare:

- **Load**: un'istruzione che legge un valore da un indirizzo di memoria e lo copia in un registro.
- **Store**: un'istruzione che scrive il contenuto di un registro in un indirizzo di memoria.

Queste operazioni sono fondamentali perché la CPU dispone di un numero limitato di registri; la maggior parte dei dati risiede in memoria e deve essere caricata quando necessario.

1.4 Il ciclo di esecuzione delle istruzioni

L'esecuzione di un programma segue il cosiddetto *instruction cycle*, composto da quattro fasi principali:

1. **Fetch**: la CPU legge dalla memoria l'istruzione all'indirizzo indicato dal PC.
2. **Decode**: l'istruzione viene interpretata, identificando l'operazione da eseguire e gli operandi coinvolti.
3. **Execute**: la CPU esegue l'operazione richiesta (aritmetica, logica, salto, accesso alla memoria, ecc.).
4. **Write-back (opzionale)**: il risultato dell'operazione viene scritto in un registro o in memoria.

Ogni fase ha un ruolo preciso e necessario. Il ciclo è ripetuto in modo continuo, costituendo il meccanismo fondamentale dell'esecuzione dei programmi.

1.5 La memoria come osservatore passivo

Un punto concettualmente cruciale è che la memoria:

- osserva unicamente una sequenza di richieste di indirizzi;
- non conosce la natura delle richieste;
- non sa se un indirizzo rappresenta codice, dati, un puntatore, una variabile o un valore numerico.

La memoria non ha alcuna informazione su *come* gli indirizzi siano stati generati, né sul significato semantico dei dati. Essa risponde semplicemente alle richieste di lettura e scrittura.

1.6 Il flusso di indirizzi come oggetto di analisi

Per l'analisi delle prestazioni, della località, delle cache e del comportamento del programma, ciò che conta è il *flusso di indirizzi* generato durante l'esecuzione.

Questo flusso è una sequenza:

$$A_1, A_2, A_3, \ldots$$

dove ogni A_i è un indirizzo di memoria richiesto dalla CPU.

Studiare questa sequenza permette di comprendere:

- la località spaziale e temporale del programma;
- l'efficienza delle cache;
- il comportamento del compilatore;
- l'impatto delle strutture dati sulla memoria.

In altre parole, l'analisi del flusso di indirizzi è uno strumento essenziale per capire come un programma interagisce con la gerarchia di memoria.

2 CPU, Memoria e il Ruolo della Cache

In questa sezione approfondiamo la relazione tra CPU, memoria principale e cache, analizzando in modo dettagliato come queste componenti interagiscono durante l'esecuzione di un programma. L'obiettivo è comprendere perché la gerarchia di memoria è strutturata in livelli, quali limiti fisici e temporali la motivano e come la cache contribuisce a ridurre i tempi di attesa della CPU.

2.1 Aree di memoria direttamente accessibili dalla CPU

La CPU può accedere direttamente soltanto a due aree di memorizzazione:

- **la memoria principale** (RAM);
- **i registri interni** della CPU.

Questa affermazione è fondamentale: la CPU non può leggere o scrivere direttamente su disco, né su dispositivi esterni, né su livelli di memoria più lenti. Ogni operazione deve passare attraverso la memoria principale o i registri.

2.2 Localizzazione delle istruzioni e dei dati

Ogni istruzione eseguita dalla CPU, insieme a tutti i dati necessari per completarla, deve trovarsi in uno dei due luoghi accessibili:

1. nei registri della CPU;
2. nella memoria principale.

Se un dato non è presente in memoria, deve essere caricato prima che la CPU possa operare su di esso. Questo implica che l'esecuzione di un programma dipende fortemente dalla disponibilità dei dati nei livelli di memoria più vicini alla CPU.

2.3 Velocità dei registri della CPU

I registri della CPU sono le unità di memoria più veloci dell'intero sistema. Essi:

- sono fisicamente integrati all'interno del processore;
- possono essere letti o scritti in un singolo ciclo di clock;
- sono progettati per supportare operazioni aritmetiche e logiche ad altissima velocità.

Molti processori moderni sono in grado di eseguire più istruzioni basate su registri in un singolo ciclo, grazie a tecniche come:
- *pipelining*; - *superscalar execution*; - *out-of-order execution.*
Queste tecniche sfruttano la rapidità dei registri per massimizzare il throughput della CPU.

2.4 Lentezza relativa della memoria principale

L'accesso alla memoria principale è significativamente più lento rispetto ai registri. Questo perché:

- la memoria si trova fisicamente più lontana dalla CPU;
- l'accesso richiede una transazione sul *memory bus*;
- tale transazione richiede più cicli di clock per essere completata.

In pratica, mentre un registro può essere letto in un ciclo, un accesso alla RAM può richiedere decine o centinaia di cicli, a seconda dell'architettura.

2.5 Stalli della CPU

Quando la CPU necessita di un dato che non è immediatamente disponibile in un registro o nella cache, deve attendere che la memoria principale completi la transazione. Questo fenomeno è chiamato:

stall della CPU

Uno *stall* rappresenta un periodo in cui la CPU non può proseguire l'esecuzione perché è in attesa di dati. Durante questo tempo, la CPU rimane inattiva, sprecando potenziale computazionale.

2.6 Introduzione della cache

Per ridurre i tempi di attesa e minimizzare gli stalli, i sistemi moderni introducono un livello intermedio di memoria: la **cache**.

La cache è:

- una memoria molto più veloce della RAM;
- fisicamente situata sul chip della CPU;
- di dimensioni ridotte rispetto alla memoria principale;
- progettata per contenere dati e istruzioni utilizzati di frequente.

La cache sfrutta i principi di:

- **località temporale**: se un dato è stato usato di recente, è probabile che venga usato di nuovo; - **località spaziale**: se un indirizzo è stato usato, è probabile che vengano usati anche indirizzi vicini.

2.7 Benefici della cache

Grazie alla cache:

- la CPU può accedere rapidamente ai dati più utilizzati;
- il numero di accessi lenti alla memoria principale diminuisce;
- gli stalli vengono ridotti;
- il throughput complessivo del sistema aumenta.

In sintesi, la cache funge da ponte tra la velocità elevatissima della CPU e la lentezza relativa della memoria principale, migliorando drasticamente le prestazioni del sistema.

3 Protezione della Memoria tramite Registri di Base e Limite

In questa sezione analizziamo un aspetto fondamentale dei sistemi operativi moderni: la protezione della memoria. Oltre alle prestazioni, un sistema deve garantire che ogni processo utente rimanga confinato nella propria area di memoria, impedendo accessi non autorizzati alla memoria del sistema operativo o di altri processi. Tale protezione è essenziale per la stabilità, la sicurezza e l'affidabilità del sistema.

3.1 La protezione della memoria come requisito di sicurezza

La protezione della memoria non riguarda soltanto la velocità o l'efficienza: essa è un requisito di sicurezza imprescindibile. Senza un meccanismo di isolamento, un processo potrebbe:

- leggere dati appartenenti ad altri utenti;
- modificare strutture interne del sistema operativo;
- corrompere la memoria di altri processi;
- compromettere l'intero sistema.

Per evitare tali scenari, ogni processo deve essere confinato nella propria regione di memoria, definita e controllata dal sistema operativo.

3.2 Perché la protezione deve essere implementata in hardware

Il sistema operativo non può verificare manualmente ogni singolo accesso alla memoria effettuato dalla CPU. Questo perché:

- la CPU esegue miliardi di istruzioni al secondo;
- ogni istruzione può generare uno o più accessi alla memoria;
- un controllo software per ogni accesso introdurrebbe un rallentamento insostenibile.

Di conseguenza, la protezione deve essere garantita direttamente dall'hardware, che può effettuare controlli in tempo reale, senza introdurre ritardi significativi.

3.3 Isolamento dei processi

Ogni processo deve rimanere confinato nella propria regione di memoria. Questo isolamento permette:

- l'esecuzione concorrente di più processi senza interferenze;
- la protezione dei dati sensibili;
- la prevenzione di errori catastrofici dovuti a puntatori errati;
- la separazione tra codice utente e codice del sistema operativo.

L'isolamento è un principio cardine dei sistemi multitasking e multiutente.

3.4 Definizione dell'intervallo di indirizzi legali

Il sistema operativo definisce, per ogni processo, quali indirizzi di memoria esso può legalmente utilizzare. Ogni riferimento alla memoria deve rientrare in questo intervallo. Se un processo tenta di accedere a un indirizzo al di fuori della propria area, l'hardware deve rilevare immediatamente la violazione e generare un'eccezione (tipicamente una *memory protection fault*).

Questo meccanismo impedisce che un processo possa, anche accidentalmente, accedere a memoria non autorizzata.

3.5 Registri hardware per la protezione: Base e Limite

La protezione della memoria viene implementata tramite due registri hardware dedicati:

- **Registro di Base (Base Register)**: contiene il più piccolo indirizzo fisico che il processo è autorizzato ad accedere.
- **Registro di Limite (Limit Register)**: contiene la dimensione della regione di memoria assegnata al processo.

Questi due registri definiscono un intervallo di indirizzi validi:

$$\text{Indirizzi validi} = [\text{Base},\ \text{Base} + \text{Limit} - 1]$$

Ogni accesso alla memoria effettuato dalla CPU viene confrontato con questo intervallo. Se l'indirizzo richiesto non rientra nei limiti, l'hardware blocca l'operazione e segnala un errore al sistema operativo.

Significato operativo dei registri

Registro di Base. Stabilisce il punto di partenza della memoria del processo. Tutti gli indirizzi logici generati dal processo vengono tradotti aggiungendo questo valore, ottenendo così l'indirizzo fisico effettivo.

Registro di Limite. Indica la lunghezza della regione di memoria concessa. Serve a verificare che l'indirizzo logico generato dal processo non ecceda la dimensione assegnata.

3.6 Funzionamento del controllo di protezione

Quando il processo effettua un accesso alla memoria:

1. genera un indirizzo logico A;
2. l'hardware verifica che $A < \text{Limit}$;
3. se il controllo è superato, l'indirizzo fisico viene calcolato come:

$$\text{Indirizzo fisico} = \text{Base} + A;$$

4. se il controllo fallisce, viene generata un'eccezione di protezione.

Questo meccanismo è estremamente efficiente perché richiede solo confronti e somme, operazioni che l'hardware può eseguire in un singolo ciclo di clock.

3.7 Importanza del meccanismo Base-Limit

Il sistema di protezione basato su registri di base e limite:

- garantisce isolamento tra processi;
- impedisce accessi non autorizzati alla memoria del sistema operativo;
- permette l'esecuzione concorrente sicura;
- costituisce la base per tecniche più avanzate come la memoria virtuale.

Nonostante la sua semplicità, questo meccanismo è stato storicamente fondamentale e rimane concettualmente importante per comprendere le architetture moderne.

4 Protezione della Memoria (Registri di Base e Limite)

In questa sezione approfondiamo il funzionamento operativo della protezione della memoria basata sui registri di *base* e *limite*, concentrandoci sul ruolo della CPU, sulle verifiche effettuate in modalità utente e sui privilegi esclusivi del sistema operativo. Questo meccanismo rappresenta uno dei primi e più importanti sistemi di protezione hardware introdotti nelle architetture di calcolo.

4.1 Controllo degli indirizzi in modalità utente

Quando un processo è in **modalità utente** (user mode), la CPU effettua un controllo su ogni indirizzo di memoria generato dal processo. Questo controllo consiste nel verificare che l'indirizzo rientri nell'intervallo definito dai registri di base e limite.

Formalmente, per ogni indirizzo logico A generato dal processo, l'hardware verifica:

$$0 \leq A < \text{Limit}$$

Se la condizione è soddisfatta, l'indirizzo fisico viene calcolato come:

$$\text{Indirizzo fisico} = \text{Base} + A$$

Questo controllo avviene *per ogni singolo accesso alla memoria*, senza eccezioni.

4.2 Gestione degli accessi illegali: trap e terminazione del processo

Se l'indirizzo generato dal processo non rientra nell'intervallo consentito, la CPU genera immediatamente una **trap**. Una trap è un'eccezione hardware che trasferisce il controllo al sistema operativo.

Il sistema operativo, ricevuta la trap, considera l'evento come una violazione della protezione della memoria e procede tipicamente a:

- terminare il processo colpevole;
- rilasciare le sue risorse;
- registrare l'evento per scopi diagnostici.

Questo comportamento impedisce che un processo possa leggere o modificare memoria non autorizzata, garantendo l'isolamento tra processi e la sicurezza del sistema.

4.3 Privilegi del sistema operativo: modifica dei registri di base e limite

Solo il **kernel** (cioè il sistema operativo in modalità privilegiata) può modificare i registri di base e limite. Questa restrizione è fondamentale per evitare che un processo utente possa alterare i propri limiti di memoria e accedere a regioni non autorizzate.

La modifica dei registri avviene tramite:

istruzioni privilegiate

che possono essere eseguite esclusivamente in modalità kernel. Se un processo utente tenta di eseguire tali istruzioni, viene generata una trap.

4.4 Accesso completo alla memoria da parte del kernel

Il kernel, a differenza dei processi utente, può accedere a **tutta** la memoria fisica del sistema. Questo privilegio è necessario per svolgere una serie di operazioni fondamentali:

- **Caricamento dei programmi**: il kernel deve poter scrivere nelle aree di memoria assegnate ai processi.
- **Gestione delle system call**: durante una chiamata di sistema, il kernel deve poter leggere e scrivere nella memoria del processo chiamante.
- **Operazioni di I/O sulla memoria utente**: ad esempio, leggere buffer forniti dal processo o scrivere risultati.
- **Generazione di memory dump**: in caso di crash, il kernel deve poter salvare l'intero contenuto della memoria per analisi post-mortem.

Queste operazioni richiedono un accesso illimitato alla memoria, motivo per cui il kernel opera in una modalità privilegiata che bypassa i controlli base/limit.

4.5 Sintesi del meccanismo di protezione

Il meccanismo basato su registri di base e limite garantisce:

- isolamento tra processi;
- protezione del sistema operativo;
- rilevamento immediato degli accessi illegali;
- semplicità ed efficienza hardware;
- un modello concettuale chiaro per la gestione della memoria.

Nonostante la sua semplicità, questo modello è stato fondamentale nella storia dei sistemi operativi e costituisce la base concettuale per tecniche più avanzate come la memoria virtuale e la paginazione.

5 Caricamento di un Programma in Memoria

In questa sezione analizziamo in dettaglio il processo mediante il quale un programma, inizialmente memorizzato come file binario su disco, viene caricato in memoria e trasformato in un processo eseguibile. Questo passaggio è fondamentale per comprendere come un sistema operativo gestisca l'esecuzione dei programmi e come ottimizzi l'uso della memoria fisica.

5.1 Il programma come file binario su disco

Un programma, prima di essere eseguito, esiste semplicemente come **file binario** memorizzato su un dispositivo di archiviazione persistente, tipicamente un disco. Questo file contiene:

- il codice macchina del programma;
- i dati iniziali;
- le informazioni necessarie al caricamento (header, tabelle, segmenti).

Il file binario non è ancora un processo: è solo una rappresentazione statica del programma.

5.2 Dal file al processo: creazione del contesto di esecuzione

Quando il sistema operativo decide di eseguire il programma, deve prima **caricarlo in memoria**. Una volta caricato, esso diventa un **processo**, cioè un'entità dotata di:

- uno spazio di indirizzamento;
- un insieme di registri (incluso il Program Counter);
- uno stack;
- un heap;
- strutture di controllo del sistema operativo (PCB, tabelle, permessi).

Questo insieme di elementi costituisce il *contesto di esecuzione* del processo.

5.3 Accesso alla memoria durante l'esecuzione

Durante l'esecuzione, il processo:

- preleva (*fetch*) le istruzioni dalla memoria;
- accede ai dati necessari per eseguire tali istruzioni;
- aggiorna il proprio stato interno.

Tutte queste operazioni avvengono tramite accessi alla memoria, che rappresenta quindi la risorsa centrale per l'esecuzione del programma.

5.4 Terminazione del processo e rilascio della memoria

Quando un processo termina — sia volontariamente, sia per errore, sia per decisione del sistema operativo — la sua area di memoria viene:

- liberata;
- restituita al sistema operativo;
- resa disponibile per altri processi.

Questo meccanismo garantisce un uso efficiente della memoria fisica e permette l'esecuzione di più programmi nel tempo.

5.5 Flessibilità nel posizionamento in memoria

Nei sistemi moderni, un processo può essere caricato **in qualunque posizione della memoria fisica**. Non è necessario che inizi all'indirizzo fisico 00000 o in una posizione fissa.

Questa flessibilità è resa possibile da:

- meccanismi di rilocazione;
- traduzione degli indirizzi;
- protezione hardware (come i registri base/limite o la memoria virtuale).

5.6 Vantaggi della flessibilità nel caricamento

La possibilità di caricare un processo in qualunque area libera della memoria fisica migliora:

- **l'utilizzo della memoria**: si riducono i buchi e la frammentazione;
- **l'efficienza del sistema**: il sistema operativo può scegliere dinamicamente la posizione migliore;
- **la scalabilità**: più processi possono coesistere senza vincoli rigidi di posizionamento.

Questa caratteristica è essenziale nei sistemi multitasking e multiutente moderni.

6 Quando Avviene il Binding degli Indirizzi

In questa sezione analizziamo il concetto di *address binding*, cioè il processo mediante il quale gli indirizzi utilizzati da un programma vengono convertiti da forme simboliche ad indirizzi fisici effettivi. Questo processo non avviene in un unico momento, ma può essere eseguito in tre fasi distinte: compilazione, caricamento ed esecuzione. Ogni fase offre diversi livelli di flessibilità e ha implicazioni importanti per la gestione della memoria.

6.1 Evoluzione degli indirizzi: da simbolici ad assoluti

Gli indirizzi che compaiono nel codice sorgente non sono immediatamente utilizzabili dalla CPU. Essi attraversano tre forme successive:

1. **Indirizzi simbolici**: nomi astratti come etichette, variabili, simboli.
2. **Indirizzi rilocabili (relocatable)**: indirizzi espressi come offset rispetto a un punto di partenza.
3. **Indirizzi assoluti**: indirizzi fisici definitivi utilizzati dall'hardware.

Il passaggio da una forma all'altra è ciò che chiamiamo *binding degli indirizzi*.

6.2 Binding durante la compilazione

Se il compilatore conosce in anticipo la posizione finale del programma in memoria, può generare direttamente **codice assoluto**. In questo caso:

- tutti gli indirizzi vengono fissati durante la compilazione;
- il programma può essere eseguito solo se viene caricato esattamente nella posizione prevista.

Un cambiamento dell'indirizzo di partenza richiede:

una ricompilazione completa

poiché tutti gli indirizzi assoluti devono essere aggiornati.

6.3 Binding durante il caricamento

Se la posizione finale del programma non è nota al momento della compilazione, il compilatore produce **codice rilocabile**. In questo caso:

- gli indirizzi sono espressi come offset;
- il loader, al momento del caricamento in memoria, calcola gli indirizzi assoluti finali;
- il programma può essere caricato in qualunque area libera della memoria fisica.

Questo approccio offre maggiore flessibilità rispetto al binding in fase di compilazione.

6.4 Binding durante l'esecuzione

Se un programma può essere spostato in memoria anche *dopo* essere stato caricato — ad esempio per compattazione, swapping o gestione dinamica della memoria — allora il binding deve essere effettuato **a run-time**.

In questo scenario:

- gli indirizzi logici vengono convertiti in indirizzi fisici durante l'esecuzione;
- l'hardware (tramite MMU, registri base/limite o paginazione) effettua la traduzione dinamica;
- il programma può essere rilocato senza necessità di ricompilazione o ricaricamento.

6.5 Motivazioni per il binding a run-time nei sistemi moderni

I sistemi operativi moderni preferiscono ritardare il binding fino al momento dell'esecuzione perché ciò permette:

- **maggiore flessibilità**: i processi possono essere spostati, sospesi, ripresi o compattati;
- **migliore gestione della memoria**: la memoria può essere utilizzata in modo più efficiente;
- **supporto alla memoria virtuale**: gli indirizzi logici non devono corrispondere a indirizzi fisici contigui;
- **isolamento e protezione**: la traduzione dinamica facilita i controlli di sicurezza.

Il binding a run-time è quindi una caratteristica essenziale delle architetture moderne, che consente una gestione avanzata e flessibile della memoria.

7 Indirizzi Logici e Indirizzi Fisici

In questa sezione analizziamo la distinzione fondamentale tra *indirizzi logici* e *indirizzi fisici*, concetto centrale per comprendere la gestione della memoria nei sistemi operativi moderni. La differenza tra queste due tipologie di indirizzi dipende dal momento in cui avviene il binding e dal modello di memoria adottato dal sistema.

7.1 Indirizzi logici generati dalla CPU

Un **indirizzo logico** (o *logical address*) è l'indirizzo prodotto dalla CPU durante l'esecuzione di un programma. Ogni volta che il programma:

- esegue un'istruzione,
- accede a una variabile,
- effettua un salto,
- legge o scrive dati,

la CPU genera un indirizzo logico.

Questo indirizzo non è necessariamente quello utilizzato dalla memoria fisica: rappresenta la visione del programma del proprio spazio di indirizzamento.

7.2 Indirizzi fisici utilizzati dall'hardware

Un **indirizzo fisico** (o *physical address*) è l'indirizzo effettivamente inviato all'hardware della memoria, tipicamente al registro MAR (Memory Address Register). È l'indirizzo che identifica una cella reale della memoria fisica.

La memoria principale non conosce indirizzi logici: essa opera esclusivamente con indirizzi fisici.

7.3 Corrispondenza tra indirizzi logici e fisici

La relazione tra indirizzi logici e fisici dipende dal momento in cui avviene il binding degli indirizzi:

- **Binding a compile time**: gli indirizzi logici coincidono con quelli fisici.
- **Binding a load time**: gli indirizzi logici vengono convertiti in fisici dal loader, quindi coincidono durante l'esecuzione.

In entrambi i casi:

$$\text{Indirizzo logico} = \text{Indirizzo fisico}$$

poiché la traduzione è già stata effettuata prima dell'esecuzione.

7.4 Binding a run-time: indirizzi logici diversi da quelli fisici

Se il binding avviene durante l'esecuzione, gli indirizzi logici non coincidono più con quelli fisici. In questo caso:

$$\text{Indirizzo logico} \neq \text{Indirizzo fisico}$$

Gli indirizzi logici vengono chiamati:

indirizzi virtuali

e la loro traduzione in indirizzi fisici è effettuata dinamicamente dall'hardware (tipicamente tramite la Memory Management Unit, MMU).

Questo meccanismo permette:

- rilocazione dinamica;
- memoria virtuale;
- protezione avanzata;
- spazi di indirizzamento separati per ogni processo.

7.5 Spazio degli indirizzi logici

Lo **spazio degli indirizzi logici** è l'insieme di tutti gli indirizzi che un programma può generare durante la sua esecuzione. Esso rappresenta la visione astratta del programma della propria memoria.

Formalmente:

$$\text{Logical Address Space} = \{0, 1, 2, \ldots, L-1\}$$

dove L è la dimensione dello spazio di indirizzamento logico.

7.6 Spazio degli indirizzi fisici

Lo **spazio degli indirizzi fisici** è l'insieme degli indirizzi realmente disponibili nella memoria fisica del sistema.

Formalmente:

$$\text{Physical Address Space} = \{0, 1, 2, \ldots, M-1\}$$

dove M è la dimensione della memoria fisica.

7.7 Importanza della distinzione

La distinzione tra indirizzi logici e fisici è fondamentale perché:

- permette ai programmi di essere indipendenti dalla memoria fisica;
- consente al sistema operativo di gestire la memoria in modo flessibile;
- abilita tecniche come la memoria virtuale, la paginazione e la segmentazione;
- garantisce isolamento e protezione tra processi.

Comprendere questa distinzione è essenziale per analizzare il funzionamento della memoria virtuale e dei moderni sistemi operativi.

8 MMU e Rilocazione degli Indirizzi

In questa sezione analizziamo il ruolo della **Memory Management Unit (MMU)** nel processo di traduzione degli indirizzi. La MMU è un componente hardware fondamentale nei sistemi moderni, poiché consente la traduzione dinamica degli indirizzi logici in indirizzi fisici durante l'esecuzione del programma. Questo meccanismo permette flessibilità, protezione e supporto alla memoria virtuale.

8.1 Funzione della Memory Management Unit

La **MMU** esegue la traduzione degli indirizzi *a run-time*, cioè mentre il programma è in esecuzione. Ogni volta che la CPU genera un indirizzo logico, la MMU lo converte immediatamente in un indirizzo fisico.

Questa traduzione avviene:

- in modo completamente trasparente al programma;
- per ogni singolo accesso alla memoria;
- senza introdurre ritardi significativi, grazie all'implementazione hardware.

Il programma non è consapevole dell'esistenza degli indirizzi fisici: esso lavora esclusivamente con indirizzi logici.

8.2 MMU semplice basata su registro di rilocazione

Una MMU elementare può essere implementata tramite un **registro di rilocazione**, che generalizza il concetto di registro di base. Questo registro contiene l'indirizzo fisico di partenza della regione di memoria assegnata al processo.

La traduzione avviene tramite la formula:

$$\text{Indirizzo fisico} = \text{Indirizzo logico} + \text{Registro di rilocazione}$$

Questa operazione è estremamente efficiente, poiché richiede solo un'addizione.

8.3 Esempio di traduzione tramite registro di rilocazione

Consideriamo un processo con:

$$\text{Registro di rilocazione} = 14000$$

Allora:

- L'indirizzo logico 0 viene tradotto in:

$$0 + 14000 = 14000$$

- L'indirizzo logico 346 viene tradotto in:

$$346 + 14000 = 14346$$

Questo esempio mostra come gli indirizzi logici siano semplicemente offset rispetto alla base fisica assegnata al processo.

8.4 Il programma vede solo indirizzi logici

Un punto fondamentale è che:

il programma non vede mai gli indirizzi fisici

Esso opera esclusivamente con indirizzi logici, che rappresentano la sua visione astratta dello spazio di memoria. La traduzione in indirizzi fisici è completamente gestita dalla MMU.

8.5 Traduzione trasparente degli indirizzi

La MMU esegue la traduzione:

- automaticamente;
- per ogni accesso alla memoria;
- senza intervento del sistema operativo durante l'esecuzione normale;
- senza che il programma debba essere modificato.

Questa trasparenza è essenziale per permettere:

- rilocazione dinamica dei processi;
- protezione della memoria;
- implementazione della memoria virtuale;
- flessibilità nella gestione dello spazio fisico.

La MMU costituisce quindi il cuore della gestione moderna della memoria.

9 Caricamento Dinamico

In questa sezione analizziamo il concetto di **caricamento dinamico** (*dynamic loading*), una tecnica che permette di migliorare l'utilizzo della memoria fisica caricando in RAM solo le parti del programma effettivamente necessarie durante l'esecuzione. Questa strategia è particolarmente utile per programmi di grandi dimensioni o contenenti routine raramente utilizzate.

9.1 Limiti dell'esecuzione tradizionale

Nel modello tradizionale di esecuzione, si assume che l'intero programma debba essere caricato in memoria fisica prima di poter essere eseguito. Questo comporta due conseguenze importanti:

- la dimensione massima del processo è limitata dalla quantità di RAM disponibile;
- anche le parti del programma mai utilizzate occupano memoria inutilmente.

Questo approccio è inefficiente, soprattutto per applicazioni complesse che includono molte routine non sempre necessarie.

9.2 Principio del caricamento dinamico

Il caricamento dinamico migliora l'utilizzo della memoria caricando in RAM solo le routine che vengono effettivamente invocate durante l'esecuzione. Il funzionamento è il seguente:

- tutte le routine del programma rimangono su disco in formato *relocatable load*;
- il programma principale viene caricato e avviato;
- quando una routine è necessaria, il programma verifica se essa è già presente in memoria;
- se non lo è, un **relocatable linking loader** la carica in memoria e aggiorna le tabelle degli indirizzi;
- l'esecuzione continua all'interno della routine appena caricata.

Questo meccanismo permette di caricare codice solo quando serve, riducendo l'occupazione di memoria.

9.3 Formato relocatable load

Le routine non caricate inizialmente rimangono su disco in un formato chiamato:

relocatable load format

che contiene:

- il codice macchina della routine;
- informazioni per la rilocazione degli indirizzi;
- tabelle dei simboli e riferimenti esterni.

Questo formato consente al loader di posizionare la routine in qualunque area libera della memoria fisica.

9.4 Esecuzione con caricamento dinamico

Il flusso operativo durante l'esecuzione è il seguente:

1. Il programma principale viene caricato e inizia l'esecuzione.
2. Quando il programma invoca una routine:
   - si verifica se la routine è già in memoria;
   - se non lo è, il loader la carica dinamicamente;
   - il loader aggiorna le tabelle degli indirizzi per riflettere la nuova posizione.
3. L'esecuzione prosegue all'interno della routine caricata.

Questo processo è trasparente al programmatore: il codice sorgente non deve essere modificato per supportare il caricamento dinamico.

9.5 Vantaggi del caricamento dinamico

Il caricamento dinamico offre numerosi benefici:

- **Riduzione dell'uso della memoria**: solo il codice effettivamente utilizzato viene caricato.
- **Maggiore efficienza**: la memoria fisica può essere utilizzata per altri processi o per dati.
- **Supporto a programmi di grandi dimensioni**: anche programmi più grandi della RAM disponibile possono essere eseguiti, purché solo una parte sia necessaria in un dato momento.
- **Particolarmente utile per routine rare**: ad esempio, routine di gestione degli errori, che vengono chiamate solo in condizioni eccezionali.

Il caricamento dinamico rappresenta quindi un passo importante verso una gestione più intelligente e flessibile della memoria.

10 Linking Dinamico e Librerie DLL

In questa sezione analizziamo il concetto di **linking dinamico** e il ruolo delle **Dynamically Linked Libraries (DLLs)**, chiamate anche *librerie condivise*. Si tratta di una tecnica fondamentale nei sistemi operativi moderni, che permette di ridurre l'uso della memoria, facilitare l'aggiornamento delle librerie e condividere codice tra più processi.

10.1 Librerie collegate dinamicamente

Le **DLL** sono librerie di sistema che vengono collegate ai programmi *durante l'esecuzione* e non in fase di compilazione. Questo significa che:

- il codice della libreria non viene copiato all'interno dell'eseguibile;
- il programma contiene solo riferimenti simbolici alle routine della libreria;
- il collegamento effettivo avviene quando il programma viene caricato o quando la routine viene invocata.

Questo approccio è diverso dal linking statico, in cui il codice della libreria viene incorporato direttamente nell'eseguibile.

10.2 Vantaggi rispetto al linking statico

Nel linking statico, ogni eseguibile contiene una copia completa delle librerie utilizzate. Questo comporta:

- aumento delle dimensioni dei file eseguibili;
- duplicazione del codice in memoria quando più processi usano la stessa libreria.

Con le DLL, invece:

- il codice della libreria non viene duplicato negli eseguibili;
- più processi possono condividere un'unica copia della libreria in memoria;
- si riduce l'uso complessivo della memoria fisica.

Per questo motivo le DLL vengono anche chiamate:

librerie condivise (shared libraries)

10.3 Funzionamento del linking dinamico

Quando un programma invoca una routine appartenente a una DLL, il processo avviene come segue:

1. Il loader verifica se la DLL è già presente in memoria.
2. Se la DLL non è ancora caricata:
   - il loader la carica in memoria;
   - viene creata o aggiornata la tabella dei riferimenti del programma.
3. I riferimenti del programma vengono aggiornati per puntare alla posizione reale della DLL in memoria.
4. L'esecuzione continua all'interno della routine della libreria.

Questo processo è completamente trasparente al programmatore.

10.4 Aggiornabilità indipendente delle DLL

Un vantaggio fondamentale delle DLL è che:

possono essere aggiornate indipendentemente dai programmi che le utilizzano

Ciò significa che:

- una correzione di bug nella DLL beneficia automaticamente tutti i programmi che la usano;
- non è necessario ricompilare o ridistribuire gli eseguibili;
- il sistema può migliorare funzionalità o sicurezza aggiornando una singola libreria.

Questa caratteristica rende le DLL uno strumento essenziale per la manutenzione e l'evoluzione dei sistemi software moderni.

10.5 Curiosità: i file `.dll` nei sistemi Windows

I file con estensione `.dll` (*Dynamic-Link Library*) nei sistemi Windows sono l'implementazione concreta del concetto di libreria dinamica descritto in questa sezione. Essi contengono:

- codice eseguibile condiviso;
- dati utilizzati da più programmi;
- tabelle di esportazione delle funzioni;
- informazioni per il linking dinamico.

Dal punto di vista operativo:

- una `.dll` viene caricata solo quando un programma ne richiede una funzione;
- se la `.dll` è già in memoria, più processi la condividono senza duplicazioni;
- il loader aggiorna i riferimenti del programma per puntare alla posizione reale della `.dll`;
- l'aggiornamento di una `.dll` aggiorna automaticamente tutti i programmi che la utilizzano.

Le `.dll` rappresentano quindi un esempio reale e molto diffuso del meccanismo di linking dinamico, e costituiscono una parte fondamentale dell'architettura dei sistemi Windows.

11 Allocazione Contigua della Memoria

In questa sezione analizziamo il modello di **allocazione contigua della memoria**, uno dei primi e più semplici schemi di gestione della memoria utilizzati nei sistemi operativi. L'obiettivo è comprendere come la memoria fisica venga suddivisa tra sistema operativo e processi utente, e come i processi vengano collocati in blocchi contigui di memoria.

11.1 Coesistenza di sistema operativo e processi utente

La memoria principale deve contenere sia:

- il **sistema operativo** (o almeno la sua parte residente);
- i **processi utente** attualmente in esecuzione.

Poiché la memoria è una risorsa limitata, essa deve essere gestita in modo efficiente per garantire che:

- il sistema operativo disponga dello spazio necessario per le sue funzioni critiche;
- il maggior numero possibile di processi utente possa risiedere in memoria contemporaneamente.

11.2 Divisione della memoria in due regioni principali

Nei sistemi tradizionali, la memoria fisica viene suddivisa in due grandi aree:

1. **Area alta della memoria**: riservata al sistema operativo residente.
2. **Area restante**: utilizzata per i processi utente.

Questa divisione garantisce che il codice del sistema operativo rimanga protetto e non venga sovrascritto dai processi utente.

11.3 Presenza simultanea di più processi

Per migliorare l'utilizzo della CPU, è fondamentale che più processi risiedano in memoria contemporaneamente. Questo permette:

- il passaggio rapido da un processo all'altro (context switch);
- la riduzione dei tempi morti della CPU;
- l'esecuzione concorrente di più programmi.

Se solo un processo potesse risiedere in memoria, la CPU rimarrebbe spesso inattiva durante operazioni di I/O.

11.4 Allocazione contigua: un processo, un blocco

Con l'**allocazione contigua**, ogni processo occupa un singolo blocco di memoria fisica, continuo e senza interruzioni. Ciò significa che:

- l'intero spazio di memoria del processo è adiacente;
- non ci sono frammenti sparsi in posizioni diverse;
- i processi sono collocati uno accanto all'altro nella memoria utente.

Questo modello è semplice da implementare, poiché richiede solo:

- un indirizzo di partenza;
- una dimensione del blocco.

11.5 Assegnazione dei blocchi ai processi

Il sistema operativo deve decidere come assegnare i blocchi di memoria ai processi in attesa nella **input queue**. Le strategie di allocazione includono:

- **first-fit**: assegnare il primo blocco libero sufficientemente grande;
- **best-fit**: assegnare il blocco libero più piccolo che possa contenere il processo;
- **worst-fit**: assegnare il blocco libero più grande disponibile.

Queste strategie influenzano:

- la frammentazione della memoria;
- l'efficienza dell'allocazione;
- la velocità di ricerca dei blocchi liberi.

L'allocazione contigua è quindi un modello semplice ma con limitazioni, che prepara il terreno per tecniche più avanzate come la paginazione e la segmentazione.

12 Protezione della Memoria con Registri di Rilocazione e Limite

In questa sezione approfondiamo un meccanismo fondamentale per la protezione della memoria nei sistemi operativi: la combinazione tra **registro di rilocazione** e **registro di limite**. Questo schema estende il modello base/limit precedentemente discusso e si integra con la MMU per garantire sicurezza, isolamento e flessibilità nella gestione della memoria.

12.1 Protezione tramite combinazione di rilocazione e limite

La protezione della memoria viene realizzata combinando due registri hardware:

- **Registro di rilocazione**: indica il più piccolo indirizzo fisico accessibile dal processo.
- **Registro di limite**: indica la dimensione massima dello spazio di indirizzi logici validi.

Questi due valori definiscono l'intervallo di memoria fisica assegnato al processo.

12.2 Intervallo di indirizzi validi

Ogni indirizzo logico A generato dal processo deve soddisfare:

$$0 \leq A < \text{Limit}$$

Se la condizione è soddisfatta, la MMU calcola l'indirizzo fisico come:

$$\text{Indirizzo fisico} = A + \text{Relocation Register}$$

Se la condizione fallisce, viene generata una trap di protezione.

12.3 Ruolo della MMU nella traduzione

La MMU:

- controlla ogni indirizzo generato dalla CPU;
- verifica che rientri nel range logico consentito;
- aggiunge il valore del registro di rilocazione per ottenere l'indirizzo fisico;
- impedisce qualsiasi accesso fuori dai limiti.

Questo controllo avviene per *ogni singolo accesso alla memoria*, garantendo protezione continua.

12.4 Caricamento dei registri durante il context switch

Durante un **context switch**, il dispatcher del sistema operativo:

- salva i registri di rilocazione e limite del processo uscente;
- carica i registri di rilocazione e limite del processo entrante.

In questo modo, ogni processo ottiene automaticamente la propria protezione della memoria, senza interferire con gli altri.

12.5 Prevenzione degli accessi illegali

Poiché ogni indirizzo generato dalla CPU viene verificato, questo meccanismo impedisce:

- accessi alla memoria del sistema operativo;
- accessi alla memoria di altri processi;
- letture o scritture accidentali fuori dallo spazio assegnato.

La protezione è quindi garantita interamente dall'hardware, senza costi aggiuntivi per il sistema operativo durante l'esecuzione normale.

12.6 Flessibilità per il sistema operativo

Un vantaggio importante di questo schema è che permette al sistema operativo di:

- modificare dinamicamente la propria occupazione di memoria;
- caricare componenti solo quando necessari (ad esempio, **device driver**);
- liberare memoria quando tali componenti non servono più.

Questo rende il sistema più efficiente e permette una gestione dinamica delle risorse.

12.7 Sintesi del meccanismo

Il modello **rilocazione + limite** offre:

- protezione hardware completa;
- isolamento tra processi;
- supporto alla rilocazione dinamica;
- flessibilità per il sistema operativo;
- semplicità di implementazione.

Si tratta di un meccanismo essenziale per comprendere la gestione della memoria nei sistemi operativi moderni.

13 Allocazione a Partizioni di Dimensione Variabile

In questa sezione analizziamo il modello di **allocazione a partizioni di dimensione variabile**, una tecnica di gestione della memoria che permette di assegnare a ciascun processo un blocco contiguo di memoria la cui dimensione dipende dalle necessità del processo stesso. Questo approccio è più flessibile rispetto alle partizioni fisse, ma introduce nuove problematiche legate alla frammentazione.

13.1 Partizioni di dimensione variabile

In questo schema, la memoria principale viene suddivisa in **partizioni di dimensione variabile**, dove:

- ogni partizione contiene esattamente un processo;
- la dimensione della partizione dipende dalla quantità di memoria richiesta dal processo;
- le partizioni non hanno dimensioni predefinite.

Questo permette un utilizzo più efficiente della memoria rispetto alle partizioni fisse, evitando sprechi dovuti a blocchi troppo grandi.

13.2 Tabella di gestione della memoria

Il sistema operativo mantiene una **tabella di allocazione** che tiene traccia di:

- quali blocchi di memoria sono occupati;
- quali blocchi sono liberi;
- la dimensione e la posizione di ogni blocco.

Questa tabella è essenziale per decidere dove collocare nuovi processi e per gestire la memoria liberata dai processi terminati.

13.3 Stato iniziale della memoria

All'avvio del sistema, tutta la memoria utente è libera e forma un unico grande blocco contiguo, chiamato:

hole (buco di memoria)

Questo grande buco iniziale verrà progressivamente suddiviso man mano che i processi vengono caricati.

13.4 Creazione di buchi durante la terminazione dei processi

Quando un processo termina:

- la sua partizione viene liberata;
- lo spazio liberato diventa un nuovo buco di memoria;
- se adiacente ad altri buchi, può essere fuso per formare un buco più grande.

Questo comportamento porta a una memoria dinamica, in cui blocchi liberi e occupati cambiano continuamente nel tempo.

13.5 Caricamento di nuovi processi nei buchi disponibili

Quando un nuovo processo deve essere caricato:

- il sistema operativo cerca un buco sufficientemente grande;
- il processo viene collocato in quel buco, occupandolo completamente o parzialmente;
- se rimane spazio inutilizzato, esso diventa un buco più piccolo.

La scelta del buco da utilizzare dipende dalla strategia di allocazione (first-fit, best-fit, worst-fit).

13.6 Frammentazione della memoria

Con il passare del tempo, la memoria tende a frammentarsi in molti blocchi liberi non contigui. Questo fenomeno è chiamato:

frammentazione esterna

La frammentazione esterna può rendere difficile trovare un blocco abbastanza grande per un nuovo processo, anche se la somma totale della memoria libera sarebbe sufficiente.

Questo è uno dei principali limiti dell'allocazione a partizioni variabili e una delle motivazioni che hanno portato allo sviluppo di tecniche più avanzate come la paginazione.

14 Gestione delle Richieste di Memoria e dei Buchi Liberi

In questa sezione analizziamo come il sistema operativo gestisce le richieste di memoria dei processi e come mantiene la lista dei buchi liberi (*free holes*) nella memoria principale. Questo meccanismo è fondamentale negli schemi di allocazione contigua e a partizioni variabili.

14.1 Valutazione delle richieste di memoria

Quando un processo arriva nel sistema, il sistema operativo:

1. esamina la quantità di memoria richiesta dal processo;
2. confronta tale richiesta con la memoria libera disponibile.

Se la memoria disponibile è sufficiente:

il processo viene caricato in memoria e può iniziare l'esecuzione

Se la memoria non è sufficiente, il sistema operativo può:

- **rifiutare** il processo, generando un errore;
- **inserirlo in una coda di attesa** finché non si libera memoria sufficiente.

14.2 Presenza di buchi di memoria di dimensioni variabili

Durante il normale funzionamento del sistema, la memoria contiene:

- blocchi occupati da processi;
- **buchi liberi** di varie dimensioni, creati dalla terminazione dei processi.

Questi buchi non sono necessariamente contigui e possono essere distribuiti in modo irregolare nella memoria.

14.3 Ricerca di un buco adatto

Quando un processo richiede memoria, il sistema operativo:

1. scorre la lista dei buchi liberi;
2. cerca un buco sufficientemente grande per contenere il processo.

Se viene trovato un buco adatto:

- il processo viene collocato in quel buco;
- se il buco è più grande del necessario, esso viene **diviso** in:
  - un blocco allocato al processo;
  - un buco libero più piccolo.

Questo meccanismo permette di utilizzare la memoria in modo flessibile, ma può aumentare la frammentazione.

14.4 Rilascio della memoria alla terminazione del processo

Quando un processo termina:

- il suo blocco di memoria viene restituito alla lista dei buchi liberi;
- il sistema operativo verifica se esistono buchi adiacenti.

Se due o più buchi sono contigui, essi vengono:

fusi in un unico buco più grande

Questo processo è chiamato **coalescing** e serve a ridurre la frammentazione esterna.

14.5 Sintesi del meccanismo di gestione

Il sistema operativo deve quindi:

- mantenere una lista aggiornata dei buchi liberi;
- scegliere un buco adatto per ogni nuova richiesta di memoria;
- dividere i buchi quando necessario;
- fondere i buchi adiacenti quando un processo termina.

Questi meccanismi sono essenziali per garantire un uso efficiente della memoria fisica negli schemi di allocazione contigua.

15 Strategie di Allocazione Dinamica della Memoria

Quando un processo richiede una quantità di memoria n, il sistema operativo deve soddisfare tale richiesta utilizzando la lista dei buchi liberi (*free holes*). Poiché i buchi hanno dimensioni variabili e sono distribuiti irregolarmente nella memoria, il sistema operativo deve scegliere quale buco utilizzare. Esistono tre politiche classiche per effettuare questa scelta.

15.1 First-fit

La strategia **first-fit** seleziona il *primo* buco libero sufficientemente grande per contenere la richiesta.

Caratteristiche principali:

- la ricerca si interrompe non appena viene trovato un buco adatto;
- è la strategia più veloce, poiché non richiede di scansionare l'intera lista;
- tende a lasciare buchi liberi nella parte iniziale della memoria.

15.2 Best-fit

La strategia **best-fit** seleziona il buco libero più piccolo che sia comunque abbastanza grande da soddisfare la richiesta.

Caratteristiche principali:

- richiede di scansionare l'intera lista dei buchi liberi;
- minimizza lo spreco immediato di memoria;
- tende a produrre molti buchi molto piccoli, difficili da riutilizzare;
- può aumentare la frammentazione esterna.

15.3 Worst-fit

La strategia **worst-fit** seleziona il buco libero più grande disponibile.

Caratteristiche principali:

- richiede anch'essa di scansionare l'intera lista;
- lascia buchi liberi di dimensioni relativamente grandi, potenzialmente più utili per future allocazioni;
- può ridurre la frammentazione in alcuni casi, ma non sempre.

15.4 Confronto tra le strategie

Le tre strategie presentano vantaggi e svantaggi:

- **First-fit** è la più veloce e spesso la più efficiente nella pratica.
- **Best-fit** minimizza lo spreco immediato ma tende a creare frammentazione fine.
- **Worst-fit** cerca di preservare buchi grandi, ma può risultare inefficiente in scenari reali.

La scelta della strategia dipende dal comportamento tipico dei processi e dal compromesso tra velocità di allocazione e frammentazione della memoria.

16 Frammentazione Esterna

In questa sezione analizziamo il fenomeno della **frammentazione esterna**, un problema tipico degli schemi di allocazione contigua e delle partizioni di dimensione variabile. La frammentazione esterna riduce l'efficienza della memoria e può impedire l'allocazione di nuovi processi anche quando la memoria totale libera sarebbe sufficiente.

16.1 Cause della frammentazione esterna

Le strategie di allocazione come **first-fit** e **best-fit** soffrono in modo significativo di frammentazione esterna. Con il tempo:

- la memoria libera si suddivide in molti piccoli buchi non contigui;
- questi buchi possono essere troppo piccoli per soddisfare nuove richieste;
- la memoria risulta inutilizzabile nonostante la quantità totale di spazio libero sia sufficiente.

16.2 Effetti della frammentazione esterna

La frammentazione esterna porta a:

- fallimento delle richieste di memoria anche quando la memoria totale libera è maggiore della richiesta;
- formazione di piccoli frammenti inutilizzabili;
- peggioramento progressivo delle prestazioni del sistema.

16.3 La regola del 50 percento

Un risultato empirico noto come:

50 percent rule

afferma che, con la strategia **first-fit**, circa un terzo della memoria totale tende a diventare inutilizzabile a causa della frammentazione esterna. Questo valore non è una legge matematica, ma una stima osservata in molti sistemi reali.

17 Frammentazione Interna ed Esterna

In questa sezione distinguiamo chiaramente tra **frammentazione esterna** e **frammentazione interna**, due fenomeni diversi ma entrambi problematici nella gestione della memoria.

17.1 Frammentazione esterna

La frammentazione esterna si verifica quando:

- esiste memoria libera complessiva sufficiente;
- ma essa è suddivisa in piccoli buchi non contigui;
- nessun buco singolo è abbastanza grande per soddisfare la richiesta.

È tipica degli schemi di allocazione contigua.

17.2 Frammentazione interna

La frammentazione interna si verifica quando:

- un processo riceve un blocco di memoria più grande della sua richiesta;
- lo spazio inutilizzato all'interno del blocco è sprecato;
- questo accade soprattutto con allocazioni a dimensione fissa.

Esempio: se un processo richiede 30 byte e il sistema assegna un blocco da 32 byte, i 2 byte rimanenti sono inutilizzabili e rappresentano frammentazione interna.

17.3 Gestione dei piccoli frammenti

Frammenti molto piccoli (ad esempio 1–2 byte) sono:

- difficili da gestire;
- inutilizzabili per nuovi processi;
- causa di spreco di memoria.

17.4 Compattazione

Una tecnica per ridurre la frammentazione esterna è la **compattazione**, che consiste nel:

- spostare i processi in memoria;
- riunire i buchi liberi in un unico grande blocco contiguo.

Tuttavia:

- la compattazione è costosa in termini di tempo;
- richiede la possibilità di rilocare i processi durante l'esecuzione;
- non è possibile se il binding avviene a run-time senza supporto hardware adeguato.

17.5 Soluzione più generale

Una soluzione più ampia consiste nell'eliminare la necessità di allocazione contigua, permettendo a un processo di occupare:

regioni di memoria non contigue

Questo approccio porta a tecniche più avanzate come:

- **paginazione**;
- **segmentazione**;
- **memoria virtuale**.

Queste tecniche risolvono in modo strutturale il problema della frammentazione esterna.

18 Introduzione alla Paginazione

La **paginazione** è una tecnica fondamentale per la gestione della memoria nei sistemi operativi moderni. Essa permette a un processo di occupare regioni di memoria fisica non contigue, superando i limiti degli schemi di allocazione contigua e risolvendo problemi strutturali come la frammentazione esterna.

18.1 Motivazioni per l'uso della paginazione

La paginazione elimina due problemi principali degli schemi di allocazione contigua:

- **frammentazione esterna**: la memoria non si spezza in piccoli buchi inutilizzabili;
- **compattazione**: non è più necessario spostare i processi per creare un grande blocco libero.

Questi vantaggi derivano dal fatto che la memoria non deve più essere assegnata in blocchi contigui.

18.2 Divisione della memoria in blocchi di dimensione fissa

La paginazione divide:

- la memoria logica del processo in blocchi di dimensione fissa chiamati **pagine**;
- la memoria fisica in blocchi della stessa dimensione chiamati **frame**.

Poiché pagine e frame hanno la stessa dimensione, una pagina può essere caricata in qualunque frame libero della memoria fisica.

18.3 Vantaggi della dimensione fissa

L'uso di blocchi di dimensione fissa offre:

- maggiore flessibilità nell'allocazione;
- riduzione della frammentazione esterna;
- gestione più semplice dei buchi liberi;
- migliore utilizzo della memoria fisica.

La frammentazione interna può esistere, ma è limitata alla parte inutilizzata dell'ultimo frame.

18.4 Adozione nei sistemi moderni

La paginazione è ampiamente utilizzata nei sistemi operativi moderni grazie alla sua:

- **efficienza**: la traduzione degli indirizzi è veloce grazie alla MMU e alle tabelle delle pagine;
- **semplicità**: la gestione della memoria è più regolare e prevedibile;
- **scalabilità**: supporta facilmente la memoria virtuale e processi di grandi dimensioni.

La paginazione rappresenta quindi la base della gestione della memoria nei sistemi contemporanei.

19 Paginazione: Meccanismo di Base

In questa sezione descriviamo in dettaglio il funzionamento della **paginazione**, analizzando come gli indirizzi logici vengano tradotti in indirizzi fisici e come pagine e frame cooperino per eliminare la necessità di allocazione contigua.

19.1 Divisione della memoria in frame e pagine

La paginazione si basa sulla divisione della memoria in blocchi di dimensione fissa:

- la **memoria fisica** è suddivisa in blocchi chiamati **frame**;
- la **memoria logica** del processo è suddivisa in blocchi della stessa dimensione chiamati **pagine**.

Poiché pagine e frame hanno la stessa dimensione, una pagina può essere caricata in *qualsiasi* frame libero.

19.2 Caricamento delle pagine in memoria

Quando un processo viene avviato:

- le sue pagine vengono caricate in frame liberi della memoria fisica;
- non è necessario che i frame siano contigui;
- il processo può essere distribuito in più punti della memoria.

Questo elimina completamente la necessità di allocazione contigua.

19.3 Memoria secondaria e dimensione dei blocchi

Anche la memoria secondaria (ad esempio il disco) è suddivisa in blocchi della stessa dimensione dei frame, o multipli di essa. Questo permette:

- il trasferimento diretto pagina–frame;
- una gestione efficiente dello swapping e della memoria virtuale.

19.4 Struttura dell'indirizzo logico

Ogni indirizzo generato dalla CPU viene suddiviso in due parti:

- **numero di pagina (p)**: indice nella tabella delle pagine;
- **offset (d)**: posizione all'interno della pagina.

Formalmente:

$$\text{Indirizzo logico} = (p, d)$$

19.5 Tabella delle pagine

Ogni processo possiede una **page table** che mappa ciascuna pagina logica al frame fisico corrispondente. La tabella contiene:

- il numero del frame in cui è caricata la pagina;
- eventuali bit di protezione o stato (valid, dirty, referenced).

19.6 Traduzione dell'indirizzo

La traduzione avviene in due passaggi:

1. La MMU usa il numero di pagina p per accedere alla tabella delle pagine e ottenere il numero del frame.
2. L'indirizzo fisico viene calcolato come:

$$\text{Indirizzo fisico} = \text{Base del frame} + d$$

L'offset d rimane invariato, poiché la pagina e il frame hanno la stessa dimensione.

19.7 Sintesi del meccanismo

Il meccanismo di base della paginazione permette:

- l'uso di memoria fisica non contigua;
- l'eliminazione della frammentazione esterna;
- una gestione più semplice e flessibile della memoria;
- un supporto naturale alla memoria virtuale.

La paginazione costituisce quindi il fondamento della gestione moderna della memoria.

20 Traduzione degli Indirizzi nella MMU

In questa sezione descriviamo in dettaglio i passaggi eseguiti dalla **Memory Management Unit (MMU)** per tradurre un indirizzo logico generato dalla CPU in un indirizzo fisico utilizzabile dalla memoria. Questo processo è fondamentale nel meccanismo di paginazione.

20.1 Generazione dell'indirizzo logico

La CPU genera un indirizzo logico che viene automaticamente suddiviso in due componenti:

- **numero di pagina (p)**: identifica quale pagina logica contiene l'indirizzo;
- **offset (d)**: indica la posizione esatta all'interno della pagina.

Formalmente:

$$\text{Indirizzo logico} = (p, d)$$

20.2 Passaggi della traduzione eseguiti dalla MMU

La MMU esegue la traduzione dell'indirizzo in quattro passaggi principali:

1. **Estrazione del numero di pagina** p dall'indirizzo logico.
2. **Accesso alla tabella delle pagine** usando p come indice.
3. **Lettura del numero di frame** f associato alla pagina p.
4. **Combinazione del frame con l'offset** d per ottenere l'indirizzo fisico.

20.3 Calcolo dell'indirizzo fisico

L'indirizzo fisico viene calcolato come:

$$\text{Indirizzo fisico} = \text{Base del frame } f + d$$

L'offset d rimane invariato, poiché la pagina e il frame hanno la stessa dimensione.

20.4 Sintesi del processo

Il processo completo può essere riassunto così:

- la CPU genera un indirizzo logico (p, d);
- la MMU usa p per trovare il frame f;
- l'indirizzo fisico finale è (f, d).

Questo meccanismo permette:

- l'uso di memoria fisica non contigua;
- una traduzione veloce e trasparente;
- protezione e isolamento tra processi;
- supporto alla memoria virtuale.

21 Dimensione della Pagina e Struttura dell'Indirizzo Logico

La dimensione della pagina (**page size**) è un parametro fondamentale nei sistemi di paginazione. Essa influenza la struttura degli indirizzi logici, la dimensione delle tabelle delle pagine, la frammentazione interna e l'efficienza complessiva del sistema.

21.1 Dimensione della pagina

La dimensione della pagina è definita dall'hardware e tipicamente è una potenza di 2. Valori comuni includono:

- 4 KB,
- 8 KB,
- 16 KB,
- 64 KB,
- 2 MB,
- 1 GB (per pagine di grandi dimensioni, *huge pages*).

L'uso di una potenza di 2 semplifica notevolmente la suddivisione dell'indirizzo logico in numero di pagina e offset.

21.2 Struttura dell'indirizzo logico

Supponiamo che:

- lo spazio degli indirizzi logici sia di dimensione 2^m;
- la dimensione della pagina sia 2^n byte.

Allora:

- i **bit meno significativi** (gli ultimi n bit) rappresentano l'**offset** d;
- i **bit più significativi** (i primi $m - n$ bit) rappresentano il **numero di pagina** p.

Formalmente:

$$\text{Indirizzo logico} = (p, d)$$

dove:

$$p = \text{bit più significativi}, \qquad d = \text{bit meno significativi}$$

21.3 Interpretazione dei campi

- **Numero di pagina (p)**: viene utilizzato come indice nella tabella delle pagine del processo.
- **Offset (d)**: indica la posizione esatta all'interno della pagina selezionata.

Poiché pagine e frame hanno la stessa dimensione, l'offset rimane invariato durante la traduzione dell'indirizzo.

21.4 Vantaggi della struttura a potenze di 2

L'uso di pagine di dimensione 2^n permette:

- una separazione semplice tramite maschere di bit;
- calcoli rapidi nella MMU;
- una traduzione efficiente degli indirizzi;
- una gestione regolare e prevedibile della memoria.

Questa struttura è uno dei motivi per cui la paginazione è estremamente efficiente nei sistemi moderni.

22 Traduzione degli Indirizzi: Esempio Numerico

In questa sezione analizziamo un esempio concreto di traduzione degli indirizzi nella paginazione, utilizzando valori specifici per la dimensione della pagina, lo spazio degli indirizzi logici e la memoria fisica. Questo esempio illustra chiaramente come la paginazione realizzi una rilocazione dinamica per ogni pagina.

22.1 Parametri del sistema

Consideriamo il seguente scenario:

- **Dimensione della pagina**: 4 byte $\Rightarrow 2^n = 4 \Rightarrow n = 2$
- **Spazio degli indirizzi logici**: 16 byte $\Rightarrow 2^m = 16 \Rightarrow m = 4$
- **Memoria fisica**: 32 byte, suddivisa in 8 frame da 4 byte ciascuno

Da questi valori ricaviamo:

$$\text{Numero di pagine logiche} = \frac{2^m}{2^n} = \frac{16}{4} = 4$$

$$\text{Numero di frame fisici} = \frac{32}{4} = 8$$

22.2 Struttura dell'indirizzo logico

Poiché $m = 4$ e $n = 2$:

- i primi $m - n = 2$ bit rappresentano il **numero di pagina** p;
- gli ultimi $n = 2$ bit rappresentano l'**offset** d.

Quindi:

$$\text{Indirizzo logico} = (p, d)$$

22.3 Rilocazione dinamica tramite paginazione

La paginazione realizza una forma di **rilocazione dinamica**, poiché:

- ogni pagina logica può essere collocata in un frame fisico arbitrario;
- la tabella delle pagine funge da "registro di rilocazione per pagina";
- la traduzione avviene a run-time per ogni accesso alla memoria.

22.4 Passaggi della traduzione

Per ogni indirizzo logico generato dalla CPU:

1. L'indirizzo viene diviso in:

$$p = \text{numero di pagina}, \qquad d = \text{offset}$$

2. La MMU usa p per accedere alla tabella delle pagine.
3. Ottiene il numero di frame f in cui si trova la pagina.
4. L'indirizzo fisico viene calcolato come:

$$\text{Indirizzo fisico} = \text{Base del frame } f + d$$

22.5 Sintesi

Questo esempio mostra che:

- la paginazione elimina la necessità di memoria contigua;
- ogni pagina può essere rilocata indipendentemente;
- la tabella delle pagine svolge il ruolo di una serie di registri di rilocazione;
- la traduzione degli indirizzi è semplice ed efficiente.

23 Frammentazione Interna nella Paginazione

La paginazione elimina completamente la frammentazione esterna, ma può introdurre una forma limitata di **frammentazione interna**. In questa sezione analizziamo quando si verifica, quanto spazio può essere sprecato e come la dimensione della pagina influisce sull'efficienza del sistema.

23.1 Assenza di frammentazione esterna

Con la paginazione:

- qualsiasi frame libero può essere utilizzato per qualsiasi pagina;
- non è necessario trovare blocchi contigui di memoria;
- la memoria non si frammenta in piccoli buchi inutilizzabili.

Questo elimina completamente la **frammentazione esterna**.

23.2 Presenza di frammentazione interna

La frammentazione interna può comunque verificarsi nell'**ultimo frame** assegnato a un processo. Se la dimensione del processo non è un multiplo esatto della dimensione della pagina:

- l'ultima pagina sarà solo parzialmente utilizzata;
- lo spazio rimanente nel frame è sprecato.

23.3 Esempio numerico

Consideriamo:

- dimensione della pagina = 2048 byte;
- dimensione del processo = 72766 byte.

Numero di pagine necessarie:

$$\left\lceil \frac{72766}{2048} \right\rceil = 36 \text{ pagine}$$

Spazio effettivamente utilizzato:

$$35 \times 2048 + (\text{parte usata dell'ultima pagina})$$

Spazio sprecato nell'ultimo frame:

$$2048 - (72766 \bmod 2048) = 962 \text{ byte}$$

23.4 Limiti teorici della frammentazione interna

- **Caso peggiore**: quasi un'intera pagina sprecata.
- **Spreco medio**: circa metà pagina per processo.

Questo valore medio deriva dal fatto che la parte inutilizzata dell'ultima pagina è distribuita uniformemente tra 0 e la dimensione della pagina.

23.5 Trade-off nella scelta della dimensione della pagina

La dimensione della pagina influisce su diversi aspetti del sistema:

- **Pagine più piccole**:
  - minore frammentazione interna;
  - tabelle delle pagine più grandi.
- **Pagine più grandi**:
  - maggiore frammentazione interna;
  - tabelle delle pagine più piccole;
  - prestazioni migliori nelle operazioni di I/O su disco.

23.6 Dimensioni tipiche delle pagine nei sistemi moderni

Nei sistemi operativi contemporanei, le dimensioni più comuni sono:

- 4 KB,
- 8 KB,

con supporto opzionale per pagine più grandi (2 MB, 1 GB) per applicazioni ad alte prestazioni.

23.7 Sintesi

La paginazione:

- elimina la frammentazione esterna;
- introduce una frammentazione interna limitata all'ultimo frame;
- richiede un compromesso nella scelta della dimensione della pagina.

24 Caricamento di un Processo in Memoria

In questa sezione analizziamo come un processo viene caricato in memoria in un sistema basato su paginazione. Il caricamento richiede la disponibilità di un numero sufficiente di frame liberi e la corretta configurazione della tabella delle pagine del processo.

24.1 Requisito di frame liberi

Un processo che richiede n pagine può essere eseguito solo se:

$$n \text{ frame liberi sono disponibili}$$

Poiché ogni pagina deve essere caricata in un frame fisico, il numero di frame disponibili determina se il processo può essere avviato.

24.2 Caricamento delle pagine nei frame

Quando il processo viene caricato:

- ogni pagina del processo viene collocata in un frame libero;
- i frame non devono essere contigui;
- per ogni pagina viene registrato il numero del frame corrispondente nella **tabella delle pagine**.

La tabella delle pagine diventa quindi la mappa che descrive dove si trovano fisicamente le pagine del processo.

24.3 Vista del programmatore

Dal punto di vista del programmatore:

- la memoria appare come un unico blocco logico continuo;
- gli indirizzi logici sono semplici numeri sequenziali;
- non c'è alcuna percezione della frammentazione o della distribuzione fisica.

La paginazione rende completamente trasparente la struttura fisica della memoria.

24.4 Vista del sistema

Dal punto di vista del sistema operativo:

- le pagine del processo sono sparse in frame fisici distribuiti ovunque nella memoria;
- la traduzione degli indirizzi logici in indirizzi fisici avviene tramite la MMU;
- il programmatore non vede nulla di questo meccanismo.

La paginazione permette quindi di separare completamente la memoria logica da quella fisica.

24.5 Responsabilità del sistema operativo

Il sistema operativo deve mantenere una struttura chiamata **frame table**, che contiene:

- lo stato di ogni frame fisico (libero o occupato);
- il processo che lo sta utilizzando;
- la pagina logica caricata in quel frame.

Questa tabella è essenziale per:

- allocare frame ai nuovi processi;
- liberare frame quando i processi terminano;
- gestire la memoria virtuale e lo swapping.

24.6 Sintesi

Il caricamento di un processo in memoria con paginazione richiede:

- un numero sufficiente di frame liberi;
- la mappatura pagina–frame nella tabella delle pagine;
- la gestione centralizzata dei frame tramite la frame table.

Il tutto avviene in modo completamente trasparente per il programmatore.

25 Page Table e Context Switching

Ogni processo in un sistema con paginazione possiede la propria **tabella delle pagine**. Questa struttura è fondamentale per la traduzione degli indirizzi logici in indirizzi fisici e deve essere gestita correttamente durante i cambi di contesto.

25.1 Tabella delle pagine per processo

Per ogni processo, il sistema operativo mantiene una tabella delle pagine separata. Un puntatore a questa tabella è memorizzato nel:

Process Control Block (PCB)

Questo permette al sistema operativo di sapere sempre quale tabella delle pagine appartiene a quale processo.

25.2 Operazioni durante il context switch

Durante un **context switch**, il sistema operativo deve:

1. salvare i registri della CPU del processo uscente;
2. ripristinare i registri della CPU del processo entrante;
3. caricare nel registro hardware:

   Page-Table Base Register (PTBR)

   il puntatore alla tabella delle pagine del nuovo processo.

Questo aggiornamento è sufficiente per cambiare completamente lo spazio degli indirizzi del processo.

25.3 Memorizzazione delle tabelle delle pagine

Le tabelle delle pagine possono essere molto grandi, quindi:

- non possono essere memorizzate nei registri della CPU;
- vengono conservate nella memoria principale;
- solo il puntatore (PTBR) risiede in un registro hardware.

Questo approccio permette di gestire tabelle di grandi dimensioni senza limitazioni hardware.

25.4 Efficienza del cambio di tabella

Il cambio di processo richiede semplicemente:

aggiornare il PTBR

Questo rende il cambio di tabella delle pagine:

- estremamente veloce;
- efficiente;
- completamente trasparente al programmatore.

25.5 Sintesi

Il sistema operativo deve:

- mantenere una tabella delle pagine per ogni processo;
- memorizzare il puntatore alla tabella nel PCB;
- aggiornare il PTBR durante ogni context switch.

Questo meccanismo permette di cambiare rapidamente lo spazio degli indirizzi attivo, garantendo isolamento e protezione tra processi.

26 Translation Lookaside Buffer (TLB)

La traduzione degli indirizzi tramite paginazione richiede, nel caso generale, due accessi alla memoria per ogni riferimento:

1. uno per leggere l'entry della tabella delle pagine;
2. uno per accedere ai dati richiesti.

Questo approccio è troppo lento per i sistemi moderni. Per accelerare la traduzione degli indirizzi, l'hardware utilizza una cache specializzata chiamata **Translation Lookaside Buffer (TLB)**.

26.1 Struttura e funzionamento della TLB

La TLB è:

- una piccola cache associativa ad alta velocità;
- contenente un sottoinsieme delle entry della tabella delle pagine;
- organizzata in coppie **(tag = numero di pagina, valore = numero di frame)**.

La ricerca nella TLB avviene direttamente nella pipeline hardware, senza costi aggiuntivi significativi.

26.2 TLB hit e TLB miss

Quando la CPU genera un indirizzo logico:

- la MMU estrae il numero di pagina p;
- cerca p nella TLB.

TLB hit:

- il numero di pagina è presente nella TLB;
- il numero di frame viene recuperato istantaneamente;
- si evita l'accesso alla tabella delle pagine in memoria;
- la traduzione è estremamente veloce.

TLB miss:

- il numero di pagina non è presente nella TLB;
- la MMU consulta la tabella delle pagine in memoria;
- l'entry recuperata viene inserita nella TLB;
- se la TLB è piena, un'altra entry deve essere sostituita.

26.3 Politiche di rimpiazzo della TLB

Quando la TLB è piena, è necessario sostituire un'entry. Le politiche comuni includono:

- **LRU (Least Recently Used)**;
- **Round-robin**;
- **Random replacement**.

Alcune entry possono essere **wired down**, cioè non sostituibili, per pagine critiche del kernel.

26.4 Dimensione della TLB

La TLB è molto più piccola della tabella delle pagine:

- tipicamente da 32 a 1024 entry;
- spesso esistono due TLB separate:
  - **Instruction TLB (I-TLB)**;
  - **Data TLB (D-TLB)**.

Questa separazione migliora ulteriormente le prestazioni.

26.5 Sintesi del ruolo della TLB

La TLB:

- memorizza un sottoinsieme delle entry della page table;
- accelera enormemente la traduzione degli indirizzi;
- riduce il numero di accessi alla memoria;
- è essenziale per ottenere prestazioni accettabili nei sistemi moderni.

27 TLB e Address Space Identifiers (ASIDs)

Per garantire una traduzione degli indirizzi corretta ed efficiente durante i cambi di contesto, molti sistemi moderni estendono la TLB con un meccanismo chiamato **Address Space Identifier (ASID)**. Gli ASID permettono alla TLB di contenere simultaneamente traduzioni appartenenti a processi diversi senza compromettere la protezione della memoria.

27.1 ASID nella TLB

Alcune TLB associano a ogni entry un **ASID**, ovvero un identificatore univoco dello spazio degli indirizzi del processo. Ogni entry della TLB contiene quindi:

- **numero di pagina** (tag);
- **numero di frame**;
- **ASID** del processo proprietario.

Una TLB hit richiede che:

numero di pagina corrispondente **e** ASID corrispondente

siano entrambi presenti.

Questo garantisce che una entry della TLB venga utilizzata solo dal processo che l'ha generata.

27.2 Vantaggi degli ASID

Gli ASID permettono:

- di mantenere nella TLB traduzioni appartenenti a processi diversi;
- di evitare conflitti tra processi;
- di ridurre drasticamente la necessità di svuotare la TLB durante i context switch.

27.3 Assenza di ASID: necessità di flush

Se l'hardware **non supporta** gli ASID:

- ad ogni context switch la TLB deve essere completamente svuotata (**TLB flush**);
- questo evita che un processo utilizzi traduzioni appartenenti a un altro processo;
- garantisce protezione e correttezza della memoria.

Il flush è necessario perché, senza ASID, la TLB non può distinguere tra entry appartenenti a processi diversi.

27.4 Sintesi

Gli ASID:

- migliorano l'efficienza dei context switch;
- permettono di mantenere entry della TLB appartenenti a più processi;
- eliminano la necessità di flush frequenti;
- garantiscono protezione e isolamento tra processi.

Senza ASID, invece:

- la TLB deve essere svuotata a ogni cambio di processo;
- si evita l'uso di traduzioni obsolete o appartenenti ad altri processi.

28 TLB Hit Ratio ed Effective Access Time (EAT)

La presenza della TLB migliora drasticamente le prestazioni della traduzione degli indirizzi. Tuttavia, il beneficio dipende dalla probabilità che una traduzione richiesta sia già presente nella TLB. Questa probabilità è chiamata **hit ratio**.

28.1 Hit ratio

L'hit ratio rappresenta la percentuale di riferimenti alla memoria per cui:

$$\text{numero di pagina} \in \text{TLB}$$

Esempio:

- hit ratio = 80% $\Rightarrow$ l'80% delle traduzioni viene trovata direttamente nella TLB.

28.2 Tempi di accesso

Consideriamo i seguenti tempi:

- accesso alla memoria: 10 ns;
- **TLB hit**: 10 ns (accesso diretto al dato);
- **TLB miss**: occorrono due accessi alla memoria:
  - 10 ns per consultare la page table;
  - 10 ns per accedere al dato;

  per un totale di 20 ns.

28.3 Effective Access Time (EAT)

L'EAT rappresenta il tempo medio di accesso alla memoria, tenendo conto della probabilità di hit e miss.

$$\text{EAT} = h \cdot t_{\text{hit}} + (1 - h) \cdot t_{\text{miss}}$$

dove:

- h = hit ratio;
- t_{hit} = tempo in caso di hit;
- t_{miss} = tempo in caso di miss.

Esempio con hit ratio = 80%

$$\text{EAT} = (0.80 \times 10) + (0.20 \times 20) = 12 \text{ ns}$$

Esempio con hit ratio = 99%

$$\text{EAT} = (0.99 \times 10) + (0.01 \times 20) = 10.1 \text{ ns}$$

28.4 Importanza di un elevato hit ratio

Un hit ratio elevato è essenziale perché:

- la TLB è molto più veloce della memoria principale;
- anche un piccolo aumento del miss ratio peggiora sensibilmente l'EAT;
- le prestazioni complessive del sistema dipendono fortemente dalla TLB.

Nei sistemi moderni, gli hit ratio tipici superano il 95% e spesso raggiungono il 99% o più, grazie a politiche di rimpiazzo efficienti e TLB separate per istruzioni e dati.

28.5 Sintesi

La TLB:

- riduce il numero di accessi alla memoria;
- migliora drasticamente l'EAT;
- richiede hit ratio elevati per garantire prestazioni ottimali.

29 Paginazione e Protezione della Memoria

La paginazione non si limita a tradurre indirizzi logici in indirizzi fisici: essa fornisce anche un meccanismo efficace di **protezione della memoria**. Ogni accesso viene verificato dalla MMU per garantire che il processo non violi i propri limiti di memoria o i permessi assegnati.

29.1 Bit di protezione nella tabella delle pagine

Ogni entry della tabella delle pagine contiene uno o più **bit di protezione**, che specificano i permessi di accesso alla pagina:

- **read** (lettura);
- **write** (scrittura);
- **execute** (esecuzione).

Questi bit vengono controllati durante la traduzione dell'indirizzo. Se un processo tenta un accesso non consentito:

trap → il sistema operativo gestisce la violazione

29.2 Bit valid / invalid

Un altro meccanismo fondamentale è il **bit valid/invalid**:

- **valid**: la pagina appartiene allo spazio degli indirizzi del processo → accesso consentito;
- **invalid**: la pagina non appartiene al processo → accesso proibito.

Un accesso a una pagina marcata come invalid genera un'eccezione immediata.

29.3 Ruolo nella protezione della memoria

Questi meccanismi garantiscono che:

- un processo non possa leggere o modificare la memoria di altri processi;
- un processo non possa accedere a pagine non allocate;
- il kernel possa proteggere le proprie pagine da accessi non autorizzati;
- eventuali errori vengano intercettati tramite trap.

La protezione è quindi integrata direttamente nella traduzione degli indirizzi.

29.4 Sintesi

La paginazione fornisce protezione della memoria tramite:

- bit di protezione (read/write/execute);
- bit valid/invalid;
- controlli hardware eseguiti dalla MMU durante ogni accesso.

Questi meccanismi assicurano che ogni processo possa accedere solo alla memoria che gli appartiene, garantendo isolamento, sicurezza e stabilità del sistema.

30 Valid–Invalid Bit: Esempio e Frammentazione Interna

La paginazione utilizza il bit **valid/invalid** per indicare se una pagina appartiene allo spazio degli indirizzi del processo. Questo meccanismo garantisce protezione e correttezza, ma può introdurre una forma particolare di frammentazione interna quando l'ultima pagina del processo non è completamente utilizzata.

30.1 Scenario dell'esempio

Consideriamo:

- spazio degli indirizzi logici: 14 bit $\Rightarrow$ intervallo 0–16383;
- il programma utilizza realmente solo gli indirizzi 0–10468;
- dimensione della pagina: 2 KB = 2048 byte.

Numero di pagine logiche:

$$\frac{16384}{2048} = 8 \text{ pagine (numerate 0–7)}$$

Numero di pagine effettivamente necessarie per il programma:

$$\left\lceil \frac{10469}{2048} \right\rceil = 6 \text{ pagine (0–5)}$$

30.2 Valid–Invalid bit

La tabella delle pagine sarà configurata così:

- pagine 0–5: **valid** $\Rightarrow$ accesso consentito;
- pagine 6–7: **invalid** $\Rightarrow$ accesso proibito, genera eccezione.

Questo impedisce al processo di accedere a indirizzi al di fuori del proprio spazio logico.

30.3 Problema: frammentazione interna nell'ultima pagina

La pagina 5 copre l'intervallo:

$$10240 - 12287$$

Il programma utilizza solo:

$$10240 - 10468$$

La parte rimanente:

$$10469 - 12287$$

è:

- marcata come **valid**;
- ma non contiene dati del programma;
- quindi è **memoria inutilizzata ma comunque accessibile**.

Questa è una forma di **frammentazione interna causata dalla paginazione**.

30.4 Perché questa area è accessibile?

Perché la paginazione lavora a livello di pagina, non di byte:

- se una pagina è necessaria anche solo parzialmente, l'intera pagina viene marcata come valid;
- la MMU non può distinguere tra byte "utili" e byte "inutilizzati" all'interno della stessa pagina.

30.5 Conseguenze

- parte della memoria assegnata al processo rimane inutilizzata;
- il processo può comunque accedervi senza violazioni;
- questo spreco è inevitabile con pagine di dimensione fissa.

30.6 Sintesi

Questo esempio mostra che:

- il bit valid/invalid protegge correttamente le pagine fuori dallo spazio del processo;
- tuttavia, l'ultima pagina può contenere memoria inutilizzata ma comunque accessibile;
- ciò rappresenta una forma di frammentazione interna tipica della paginazione.

31 Paginazione e Codice Condiviso Reentrant

La paginazione permette non solo di isolare gli spazi di indirizzamento dei processi, ma anche di condividere in modo sicuro alcune pagine tra più processi. Questo è particolarmente utile per il codice delle librerie di sistema, che viene utilizzato da molti processi contemporaneamente.

31.1 Codice reentrant

Il codice condivisibile deve essere **reentrant**, ovvero:

- non deve essere auto-modificante;
- non deve contenere dati globali modificabili condivisi tra processi;
- deve essere sicuro da eseguire simultaneamente da più processi.

Esempi tipici includono:

- librerie standard (come `libc`);
- funzioni di sistema;
- codice del kernel marcato come condivisibile.

31.2 Condivisione delle pagine

Con la paginazione:

- una singola copia fisica del codice reentrant viene mantenuta in memoria;
- ogni processo mappa le pagine condivise nella propria tabella delle pagine;
- il codice appare a ciascun processo come parte del proprio spazio logico.

La condivisione è completamente trasparente per i processi.

31.3 Vantaggi della condivisione

La condivisione del codice reentrant permette:

- un enorme risparmio di memoria fisica;
- una riduzione del tempo di caricamento dei processi;
- una maggiore efficienza della cache;
- una gestione più semplice delle librerie comuni.

31.4 Esempio numerico

Supponiamo che:

- una libreria condivisa occupi 2 MB;
- 40 processi la utilizzino contemporaneamente.

Senza condivisione:

$$40 \times 2 \text{ MB} = 80 \text{ MB}$$

Con la paginazione e il codice reentrant:

$$\text{Uso totale} = 2 \text{ MB}$$

Il risparmio è enorme, soprattutto nei sistemi con molti processi attivi.

31.5 Sintesi

La paginazione permette la condivisione sicura del codice reentrant:

- una sola copia fisica del codice è mantenuta in memoria;
- ogni processo la mappa tramite la propria page table;
- si ottengono grandi risparmi di memoria e migliori prestazioni.

Questo meccanismo è fondamentale nei sistemi operativi moderni.

32 Paginazione Gerarchica (Two-Level Paging)

La paginazione a due livelli viene utilizzata per gestire spazi di indirizzamento molto grandi, nei quali una singola tabella delle pagine sarebbe troppo estesa per essere mantenuta interamente in memoria. L'idea è suddividere la tabella delle pagine in strutture più piccole, allocate solo quando necessario.

32.1 Motivazione

Consideriamo un sistema con:

- indirizzi logici a 32 bit;
- dimensione della pagina pari a 4 KB = 2^{12} byte.

Il numero di pagine logiche è:

$$\frac{2^{32}}{2^{12}} = 2^{20} = 1\,048\,576$$

Una tabella delle pagine con un milione di entry sarebbe troppo grande per essere mantenuta interamente in memoria per ogni processo.

32.2 Suddivisione dell'indirizzo logico

La soluzione è dividere la tabella delle pagine in due livelli. L'indirizzo logico viene suddiviso in tre campi:

$$\underbrace{p_1}_{10\text{ bit}} \quad \underbrace{p_2}_{10\text{ bit}} \quad \underbrace{d}_{12\text{ bit}}$$

dove:

- p_1: indice nella **outer page table**;
- p_2: indice nella **inner page table** selezionata da p_1;
- d: offset all'interno della pagina.

32.3 Struttura della paginazione a due livelli

La tabella delle pagine viene suddivisa così:

- una **outer page table** con $2^{10} = 1024$ entry;
- ciascuna entry punta a una **inner page table** contenente altre 1024 entry;
- ogni inner table mappa 1024×4 KB = 4 MB di spazio logico.

32.4 Allocazione su richiesta

Un vantaggio fondamentale è che:

solo le inner page table effettivamente necessarie vengono allocate

Se un processo utilizza solo una piccola parte del proprio spazio logico:

- molte inner table non vengono create;
- si risparmia una grande quantità di memoria.

32.5 Vantaggi della paginazione gerarchica

- riduce drasticamente la memoria necessaria per le page table;
- permette di gestire spazi di indirizzamento molto grandi;
- mantiene la traduzione degli indirizzi efficiente e strutturata;
- evita di allocare una tabella delle pagine monolitica e enorme.

32.6 Sintesi

La paginazione a due livelli:

- divide la tabella delle pagine in strutture più piccole;
- utilizza un indirizzo logico suddiviso in p_1, p_2, d;
- alloca solo le inner table necessarie;
- consente notevoli risparmi di memoria rispetto a una page table unica.

33 Paginazione Gerarchica nei Sistemi a 64 Bit

La paginazione gerarchica a due livelli è sufficiente per gli spazi di indirizzamento a 32 bit, ma diventa rapidamente impraticabile nei sistemi a 64 bit. In questa sezione analizziamo i limiti della paginazione tradizionale e la necessità di introdurre più livelli di tabelle.

33.1 Dimensione dello spazio logico a 64 bit

Uno spazio degli indirizzi a 64 bit contiene:

$$2^{64} \text{ indirizzi}$$

Con pagine da 4 KB (2^{12} byte), il numero di pagine logiche è:

$$\frac{2^{64}}{2^{12}} = 2^{52}$$

Una page table con 2^{52} entry è impossibile da gestire come struttura monolitica.

33.2 Limiti della paginazione a due livelli

Se si tenta di estendere la paginazione a due livelli:

$$p_1 = 42 \text{ bit}, \quad p_2 = 10 \text{ bit}, \quad d = 12 \text{ bit}$$

La outer page table avrebbe:

$$2^{42} \text{ entry}$$

Dimensione:

$$2^{42} \times 8 \text{ byte} = 2^{45} \text{ byte} = 32 \text{ TB}$$

Assolutamente impossibile da mantenere in memoria.

33.3 Aggiungere più livelli di paginazione

La soluzione è aumentare il numero di livelli della gerarchia:

- si suddivide l'indirizzo logico in più campi;
- ogni campo seleziona una tabella più piccola;
- solo le tabelle necessarie vengono allocate.

Questo migliora:

- flessibilità;
- efficienza della memoria;
- scalabilità.

33.4 Esempio: paginazione a 3 livelli

Con pagine da 4 KB, una possibile suddivisione è:

$$p_1 = 32 \text{ bit}, \quad p_2 = 10 \text{ bit}, \quad p_3 = 10 \text{ bit}, \quad d = 12 \text{ bit}$$

Tuttavia, anche in questo caso:

$$\text{outermost table size} = 2^{32} \times 8 \text{ byte} = 2^{35} \text{ byte} \approx 34 \text{ GB}$$

Ancora troppo grande per essere mantenuta in memoria.

33.5 Conclusione: la paginazione gerarchica non basta

La paginazione gerarchica tradizionale:

- funziona bene per 32 bit;
- non è scalabile per 64 bit;
- richiede troppa memoria per le tabelle esterne;
- necessita di ulteriori ottimizzazioni.

Per questo motivo, i sistemi moderni (come x86-64) utilizzano:

- 4 o 5 livelli di paginazione;
- indirizzi virtuali "ridotti" (canonical form);
- allocazione su richiesta delle tabelle intermedie.

33.6 Sintesi

Nei sistemi a 64 bit:

- la paginazione a due livelli è impraticabile;
- anche tre livelli non sono sufficienti;
- servono più livelli per mantenere le tabelle gestibili;
- solo le parti necessarie della gerarchia vengono allocate;
- questo garantisce scalabilità ed efficienza.

34 Hashed Page Tables

Le **hashed page tables** vengono utilizzate nei sistemi con spazi di indirizzamento molto grandi, tipicamente a 64 bit, dove le page table tradizionali (anche gerarchiche) diventano troppo grandi o inefficienti. L'idea è utilizzare una struttura basata su hashing per individuare rapidamente la corrispondenza tra pagina virtuale e frame fisico.

34.1 Motivazione

In uno spazio logico a 64 bit:

$$2^{52} \text{ pagine logiche (con pagine da 4 KB)}$$

Una page table tradizionale sarebbe enorme. Le hashed page tables riducono drasticamente la memoria necessaria e accelerano la ricerca.

34.2 Struttura della hashed page table

La hashed page table contiene:

- una **tabella hash** indicizzata tramite il numero di pagina virtuale;
- per ogni posizione della tabella, una **lista concatenata** di entry che gestisce eventuali collisioni.

Ogni nodo della lista contiene:

- il **numero di pagina virtuale**;
- il **numero di frame fisico** corrispondente;
- un **puntatore** al nodo successivo.

34.3 Procedura di lookup

Quando la CPU genera un indirizzo logico:

1. la MMU estrae il numero di pagina virtuale;
2. calcola la funzione hash:

$$h = \text{hash(virtual page number)}$$

3. accede alla lista nella posizione h;
4. scorre la lista finché non trova una entry con lo stesso numero di pagina;
5. recupera il frame fisico associato.

Se la pagina non è presente nella lista, si verifica un page fault.

34.4 Gestione delle collisioni

Poiché più numeri di pagina possono produrre lo stesso valore hash, le collisioni vengono gestite tramite:

- **liste concatenate** (chaining);
- ricerca lineare nella lista fino a trovare la entry corretta.

Questo approccio è semplice ed efficace anche per spazi di indirizzamento molto grandi.

34.5 Vantaggi delle hashed page tables

- adatte a spazi virtuali enormi (64 bit);
- richiedono memoria proporzionale alle pagine effettivamente utilizzate;
- lookup mediamente molto veloce;
- struttura flessibile e dinamica.

34.6 Sintesi

Le hashed page tables:

- utilizzano una funzione hash per mappare pagine virtuali a frame fisici;
- gestiscono le collisioni tramite liste concatenate;
- sono ideali per sistemi con spazi di indirizzamento molto grandi;
- riducono drasticamente la memoria necessaria rispetto alle page table tradizionali.

35 Inverted Page Tables

Le **inverted page tables** rappresentano un approccio alternativo alla gestione della memoria virtuale, progettato per ridurre drasticamente la dimensione delle page table nei sistemi con spazi di indirizzamento molto grandi. A differenza delle page table tradizionali, che hanno una entry per ogni pagina virtuale, la inverted page table ha una entry per ogni **frame fisico**.

35.1 Caratteristiche principali

Una inverted page table:

- è unica per l'intero sistema;
- contiene una entry per ogni frame fisico, non per ogni pagina virtuale;
- riduce la memoria necessaria per la gestione delle page table;
- è particolarmente utile nei sistemi a 64 bit.

35.2 Contenuto di ogni entry

Ogni entry della inverted page table contiene:

- **PID** (Process ID): identifica il processo proprietario della pagina;
- **numero di pagina virtuale** del processo;
- **numero del frame fisico** (implicito, dato dalla posizione dell'entry);
- un eventuale **puntatore** per gestire collisioni (hashing).

35.3 Procedura di traduzione degli indirizzi

Quando la CPU genera un indirizzo logico, esso contiene:

$$\text{PID} + p + d$$

dove:

- p: numero di pagina virtuale;
- d: offset all'interno della pagina.

La traduzione avviene così:

1. la MMU cerca nella inverted page table una entry con:

$$\text{PID corrispondente} \quad \text{e} \quad p \text{ corrispondente}$$

2. quando trova la entry, ottiene il numero del frame fisico i;
3. l'indirizzo fisico finale è:

$$\text{frame } i + d$$

35.4 Ricerca tramite hashing

Poiché la ricerca lineare sarebbe troppo lenta, la inverted page table è quasi sempre implementata con:

- una **funzione hash** applicata al numero di pagina virtuale;
- liste concatenate per gestire collisioni.

Questo approccio è simile alle hashed page tables, ma con una struttura globale anziché per-processo.

35.5 Vantaggi delle inverted page tables

- riducono drasticamente la memoria necessaria per le page table;
- la dimensione della tabella dipende dalla memoria fisica, non da quella virtuale;
- sono ideali per sistemi con spazi virtuali enormi (64 bit);
- garantiscono protezione tramite l'associazione PID + pagina.

35.6 Limiti

- la ricerca può essere più lenta rispetto alle page table tradizionali;
- richiede hashing efficiente per mantenere buone prestazioni;
- non è ideale per sistemi con TLB molto piccole.

35.7 Sintesi

Le inverted page tables:

- utilizzano una entry per ogni frame fisico;
- associano a ogni entry un PID e un numero di pagina virtuale;
- riducono notevolmente la memoria necessaria per la gestione della memoria virtuale;
- sono particolarmente adatte ai sistemi con grandi spazi di indirizzamento.

36 Swapping

Lo **swapping** è una tecnica utilizzata dai sistemi operativi per aumentare il grado di multiprogrammazione e permettere l'esecuzione di più processi rispetto alla memoria fisica disponibile. Consiste nello spostare temporaneamente un processo dalla memoria principale alla memoria secondaria.

36.1 Principio di funzionamento

Un processo può essere eseguito solo se si trova nella memoria principale. Tuttavia:

- quando la memoria è insufficiente,
- oppure quando un processo è inattivo per un certo periodo,

il sistema operativo può decidere di:

swap out: spostare il processo su memoria secondaria

Successivamente, quando il processo deve riprendere l'esecuzione:

swap in: riportarlo in memoria principale

36.2 Vantaggi dello swapping

Lo swapping permette:

- di superare i limiti della RAM fisica;
- di aumentare il numero di processi residenti nel sistema;
- di liberare memoria quando necessario;
- di migliorare il grado di multiprogrammazione.

In altre parole, la memoria totale "apparente" del sistema può superare la memoria fisica disponibile.

36.3 Relazione con la multiprogrammazione

Grazie allo swapping:

- processi inattivi o in attesa possono essere rimossi dalla RAM;
- lo spazio liberato può essere assegnato a processi attivi;
- il sistema può mantenere più processi pronti all'esecuzione.

Questo aumenta l'utilizzo della CPU e migliora le prestazioni complessive del sistema.

36.4 Sintesi

Lo swapping:

- sposta processi tra memoria principale e secondaria;
- permette di eseguire più processi di quanti la RAM possa contenere;
- abilita un elevato grado di multiprogrammazione;
- è un meccanismo fondamentale nei sistemi operativi tradizionali e moderni.

37 Standard Swapping

Lo **standard swapping** è la forma tradizionale di swapping utilizzata dai sistemi operativi classici. In questo modello, l'unità di scambio non è la pagina o il frame, ma l'intero processo. Il sistema operativo può spostare un processo completo fuori dalla memoria principale per liberare spazio per altri processi più attivi.

37.1 Caratteristiche dello standard swapping

Lo standard swapping:

- sposta **interi processi** tra RAM e memoria secondaria;
- richiede di salvare tutto lo stato del processo:
  - codice,
  - dati,
  - stack,
  - thread,
  - metadati del PCB.
- libera una quantità significativa di memoria;
- è utile quando la RAM è insufficiente per mantenere tutti i processi attivi.

37.2 Processi ideali per lo swap out

Il sistema operativo tende a scegliere per lo swap out:

- processi **inattivi** o in attesa di eventi lenti (I/O, timer);
- processi **a bassa priorità**;
- processi che consumano molta memoria ma non sono immediatamente necessari.

Questo minimizza l'impatto sulle prestazioni complessive del sistema.

37.3 Swap in

Quando un processo deve riprendere l'esecuzione:

- viene ricaricato dalla memoria secondaria alla RAM;
- il suo stato viene ripristinato;
- il processo torna nella coda dei processi pronti.

Il tempo di swap in/out può essere significativo, poiché coinvolge accessi al disco.

37.4 Vantaggi dello standard swapping

- permette di aumentare la multiprogrammazione;
- consente di superare i limiti della memoria fisica;
- libera grandi quantità di RAM in un'unica operazione;
- è semplice da implementare.

37.5 Limiti

- operazioni di swap molto lente (dipendono dal disco);
- non adatto a sistemi con processi molto grandi;
- può causare ritardi percepibili se usato frequentemente.

37.6 Sintesi

Lo standard swapping:

- sposta interi processi tra RAM e memoria secondaria;
- è efficace per liberare memoria rapidamente;
- è ideale per processi inattivi o a bassa priorità;
- rappresenta il modello classico di swapping nei sistemi operativi.

38 Paged Swapping (Swapping Moderno)

I sistemi operativi moderni (come Linux e Windows) non utilizzano più lo *standard swapping* basato sullo spostamento dell'intero processo. Al contrario, adottano un meccanismo molto più efficiente chiamato **paged swapping**, in cui l'unità di scambio è la singola pagina di memoria.

38.1 Caratteristiche del paged swapping

Nel paged swapping:

- non viene spostato l'intero processo;
- solo le pagine non immediatamente necessarie vengono trasferite su memoria secondaria;
- il processo rimane logicamente in esecuzione, con il suo stato completamente preservato.

Questo approccio è molto più flessibile e veloce rispetto allo swapping tradizionale.

38.2 Operazioni fondamentali

Page Out:

- una pagina viene rimossa dalla RAM;
- viene salvata nello *swap space* su disco;
- libera memoria fisica immediatamente.

Page In:

- una pagina precedentemente spostata su disco viene ricaricata in RAM;
- avviene quando il processo tenta di accedere a quella pagina;
- è gestito tramite page fault.

38.3 Vantaggi del paged swapping

Il paged swapping offre numerosi benefici:

- è molto più rapido dello swapping di interi processi;
- permette di liberare memoria in modo fine e selettivo;
- mantiene intatto lo stato del processo;
- evita lunghi tempi di attesa per lo swap in/out completo;
- supporta un elevato grado di multiprogrammazione.

38.4 Perché è preferito nei sistemi moderni

I processi moderni possono occupare centinaia di MB o diversi GB. Spostare l'intero processo sarebbe:

- troppo lento;
- inefficiente;
- dannoso per le prestazioni del sistema.

Il paged swapping consente invece di:

- spostare solo le pagine realmente inattive;
- mantenere in RAM solo ciò che è necessario;
- reagire dinamicamente alla pressione di memoria.

38.5 Sintesi

Il paged swapping:

- sposta singole pagine, non interi processi;
- è più veloce e flessibile dello swapping tradizionale;
- libera RAM quando necessario;
- preserva completamente lo stato del processo;
- è il meccanismo di swapping utilizzato nei sistemi operativi moderni.

39 Memory Management nei Sistemi Operativi Mobile

I sistemi operativi mobile (come iOS e Android) adottano strategie di gestione della memoria molto diverse rispetto ai sistemi desktop. Una differenza fondamentale è che **non utilizzano lo swapping**. Le ragioni sono legate sia alle caratteristiche dell'hardware mobile sia agli obiettivi di efficienza energetica e durata del dispositivo.

39.1 Perché i sistemi mobile non usano lo swapping

I dispositivi mobile utilizzano memoria flash (SSD integrati), che presenta due limiti importanti:

- **latenza elevata** rispetto alla RAM;
- **numero limitato di cicli di scrittura**.

Lo swapping richiederebbe continui trasferimenti di pagine tra RAM e memoria flash, causando:

- rallentamenti significativi;
- usura accelerata della memoria;
- riduzione della durata del dispositivo.

Per questo motivo, i sistemi mobile adottano strategie alternative.

39.2 iOS: gestione della memoria

iOS utilizza un modello molto aggressivo e controllato dal sistema operativo:

- richiede alle app di liberare memoria non utilizzata;
- elimina i dati **read-only** (come codice e risorse immutabili), che possono essere ricaricati dal file system;
- se la memoria rimane insufficiente, il sistema **termina l'app** per proteggere la stabilità del sistema.

Le app devono essere progettate per:

- rispondere rapidamente ai segnali di memoria bassa;
- salvare lo stato in modo da poter essere riaperte senza perdita di dati.

39.3 Android: gestione della memoria

Android utilizza un modello simile, ma con un approccio più flessibile:

- quando la memoria è bassa, termina le app in background;
- salva lo stato dell'app per permettere un **resume** rapido;
- utilizza un *Low Memory Killer* (o meccanismi equivalenti) per decidere quali app chiudere.

Le app Android devono:

- gestire correttamente il ciclo di vita (onPause, onStop, onDestroy);
- evitare sprechi di memoria;
- rilasciare risorse quando non necessarie.

39.4 Conseguenze per gli sviluppatori

Poiché non esiste swapping:

- la memoria deve essere gestita con estrema attenzione;
- l'uso inefficiente della RAM porta a chiusure improvvise delle app;
- è fondamentale ottimizzare l'uso di immagini, cache e strutture dati.

39.5 Sintesi

Nei sistemi mobile:

- lo swapping non è utilizzato per limiti hardware e per preservare la memoria flash;
- iOS libera memoria e termina app se necessario;
- Android chiude app in background e salva lo stato per un resume rapido;
- gli sviluppatori devono gestire la memoria in modo efficiente per garantire prestazioni e stabilità.

40 Esercizio 1: Bit dell'indirizzo logico e fisico

40.1 Testo

Considera uno spazio di indirizzi logici composto da 64 pagine, ciascuna delle quali contiene 1024 parole, mappato su una memoria fisica di 32 frame. Domanda:

- Quanti bit servono per l'indirizzo logico?
- Quanti bit servono per l'indirizzo fisico?

40.2 Passo 1: Interpretare i dati

- **64 pagine logiche**: significa che lo spazio logico è suddiviso in 64 blocchi (pagine).
- **1024 parole per pagina**: ogni pagina contiene 1024 indirizzi distinti (offset).
- **32 frame fisici**: la memoria fisica è suddivisa in 32 blocchi (frame).

In un sistema paginato, un indirizzo è sempre scomposto in:

$$\text{indirizzo} = \text{numero di pagina/frame} + \text{offset}$$

40.3 Passo 2: Calcolare i bit per il numero di pagina logica

Abbiamo 64 pagine logiche.

$$64 = 2^6$$

Per rappresentare 64 valori distinti (da 0 a 63) servono 6 bit.

- **Perché?** Perché con n bit posso rappresentare 2^n combinazioni diverse.
- Qui voglio almeno 64 combinazioni $\rightarrow$ cerco n tale che $2^n \geq 64$.
- $2^6 = 64$, quindi bastano 6 bit.

$$\Rightarrow \text{numero di pagina logica} = 6 \text{ bit}$$

40.4 Passo 3: Calcolare i bit per l'offset

Ogni pagina contiene 1024 parole.

$$1024 = 2^{10}$$

Per indirizzare 1024 posizioni all'interno della pagina (offset da 0 a 1023) servono 10 bit.

- **Perché?** Stesso ragionamento: cerco n tale che $2^n \geq 1024$.
- $2^{10} = 1024$, quindi servono 10 bit.

$$\Rightarrow \text{offset} = 10 \text{ bit}$$

40.5 Passo 4: Comporre l'indirizzo logico

Un indirizzo logico è:

$$\text{indirizzo logico} = \text{numero di pagina logica} + \text{offset}$$

Quindi:

$$\text{bit indirizzo logico} = 6 + 10 = 16$$

$$\Rightarrow \boxed{\text{indirizzo logico: 16 bit}}$$

40.6 Passo 5: Calcolare i bit per il numero di frame fisico

Ora guardiamo la memoria fisica: abbiamo 32 frame.

$$32 = 2^5$$

Per rappresentare 32 frame distinti (da 0 a 31) servono 5 bit.

- **Perché?** Cerco n tale che $2^n \geq 32$.
- $2^5 = 32$, quindi bastano 5 bit.

$$\Rightarrow \text{numero di frame fisico} = 5 \text{ bit}$$

40.7 Passo 6: Comporre l'indirizzo fisico

L'offset non cambia: una volta che sono dentro una pagina/frame, il numero di parole è lo stesso (1024), quindi:

$$\text{offset} = 10 \text{ bit}$$

L'indirizzo fisico è:

$$\text{indirizzo fisico} = \text{numero di frame} + \text{offset}$$

Quindi:

$$\text{bit indirizzo fisico} = 5 + 10 = 15$$

$$\Rightarrow \boxed{\text{indirizzo fisico: 15 bit}}$$

40.8 Riepilogo Esercizio 1

- **Bit indirizzo logico**: 16 bit (6 per la pagina, 10 per l'offset).
- **Bit indirizzo fisico**: 15 bit (5 per il frame, 10 per l'offset).

41 Esercizio 2: First-Fit, Best-Fit, Worst-Fit

41.1 Testo

Si hanno:

- **Partizioni** (in ordine): 300, 600, 350, 200, 750, 125 KB.
- **Processi** (in ordine): 115, 500, 358, 200, 375 KB.

Domanda: come vengono allocati i processi usando:

- first-fit,
- best-fit,
- worst-fit?

41.2 Passo 1: Riscrivere i dati in forma strutturata

- Partizioni:

$$P_1 = 300,\ P_2 = 600,\ P_3 = 350,\ P_4 = 200,\ P_5 = 750,\ P_6 = 125 \text{ KB}$$

- Processi:

$$A = 115,\ B = 500,\ C = 358,\ D = 200,\ E = 375 \text{ KB}$$

41.3 First-Fit: passo per passo

Idea: per ogni processo, scorro le partizioni dall'inizio e prendo la *prima* che è abbastanza grande.

Processo A = 115 KB

Scorro le partizioni in ordine:

- P1 = 300 KB: $300 \geq 115 \rightarrow$ **può contenere A**. $\Rightarrow$ assegno A a P1.

Aggiorno P1:

$$P_1 : 300 - 115 = 185 \text{ KB liberi}$$

Processo B = 500 KB

Scorro di nuovo dall'inizio:

- P1 = 185 KB: $185 < 500 \rightarrow$ non basta.
- P2 = 600 KB: $600 \geq 500 \rightarrow$ **può contenere B**. $\Rightarrow$ assegno B a P2.

Aggiorno P2:

$$P_2 : 600 - 500 = 100 \text{ KB liberi}$$

Processo C = 358 KB

Scorro dall'inizio:

- P1 = 185 KB: $185 < 358 \rightarrow$ no.
- P2 = 100 KB: $100 < 358 \rightarrow$ no.
- P3 = 350 KB: $350 < 358 \rightarrow$ no.
- P4 = 200 KB: $200 < 358 \rightarrow$ no.
- P5 = 750 KB: $750 \geq 358 \rightarrow$ **può contenere C**. $\Rightarrow$ assegno C a P5.

Aggiorno P5:

$$P_5 : 750 - 358 = 392 \text{ KB liberi}$$

Processo D = 200 KB

Scorro dall'inizio:

- P1 = 185 KB: $185 < 200 \rightarrow$ no.
- P2 = 100 KB: $100 < 200 \rightarrow$ no.
- P3 = 350 KB: $350 \geq 200 \rightarrow$ **può contenere D**. $\Rightarrow$ assegno D a P3.

Aggiorno P3:

$$P_3 : 350 - 200 = 150 \text{ KB liberi}$$

Processo E = 375 KB

Scorro dall'inizio:

- P1 = 185 KB: $185 < 375 \rightarrow$ no.
- P2 = 100 KB: $100 < 375 \rightarrow$ no.
- P3 = 150 KB: $150 < 375 \rightarrow$ no.
- P4 = 200 KB: $200 < 375 \rightarrow$ no.
- P5 = 392 KB: $392 \geq 375 \rightarrow$ **può contenere E**. $\Rightarrow$ assegno E a P5.

Aggiorno P5:

$$P_5 : 392 - 375 = 17 \text{ KB liberi}$$

Riepilogo First-Fit

- P1: A (115) + 185 liberi
- P2: B (500) + 100 liberi
- P3: D (200) + 150 liberi
- P4: 200 liberi
- P5: C (358) + E (375) + 17 liberi
- P6: 125 liberi

Tutti i processi sono stati allocati.

41.4 Best-Fit: passo per passo

Idea: per ogni processo, scelgo la *più piccola* partizione che sia comunque abbastanza grande.

Stato iniziale

$$P_1 = 300,\ P_2 = 600,\ P_3 = 350,\ P_4 = 200,\ P_5 = 750,\ P_6 = 125$$

Processo A = 115 KB

Partizioni che possono contenere 115 KB:

$$300,\ 600,\ 350,\ 200,\ 750,\ 125$$

La più piccola tra queste è 125 KB (P6).

$$\Rightarrow \text{assegno A a P6}$$

Aggiorno P6:

$$P_6 : 125 - 115 = 10 \text{ KB liberi}$$

Processo B = 500 KB

Partizioni ancora disponibili:

$$P_1 = 300,\ P_2 = 600,\ P_3 = 350,\ P_4 = 200,\ P_5 = 750,\ P_6 = 10$$

Partizioni che possono contenere 500 KB:

$$600,\ 750$$

La più piccola è 600 (P2).

$$\Rightarrow \text{assegno B a P2}$$

Aggiorno P2:

$$P_2 : 600 - 500 = 100 \text{ KB liberi}$$

Processo C = 358 KB

Partizioni disponibili:

$$P_1 = 300,\ P_2 = 100,\ P_3 = 350,\ P_4 = 200,\ P_5 = 750,\ P_6 = 10$$

Partizioni che possono contenere 358 KB:

$$750 \text{ (solo P5)}$$

$$\Rightarrow \text{assegno C a P5}$$

Aggiorno P5:

$$P_5 : 750 - 358 = 392 \text{ KB liberi}$$

Processo D = 200 KB

Partizioni disponibili:

$$P_1 = 300,\ P_2 = 100,\ P_3 = 350,\ P_4 = 200,\ P_5 = 392,\ P_6 = 10$$

Partizioni che possono contenere 200 KB:

$$300,\ 350,\ 200,\ 392$$

La più piccola è 200 (P4).

$$\Rightarrow \text{assegno D a P4}$$

Aggiorno P4:

$$P_4 : 200 - 200 = 0 \text{ KB liberi}$$

Processo E = 375 KB

Partizioni disponibili:

$$P_1 = 300,\ P_2 = 100,\ P_3 = 350,\ P_4 = 0,\ P_5 = 392,\ P_6 = 10$$

Partizioni che possono contenere 375 KB:

$$392 \text{ (solo P5)}$$

$$\Rightarrow \text{assegno E a P5}$$

Aggiorno P5:

$$P_5 : 392 - 375 = 17 \text{ KB liberi}$$

Riepilogo Best-Fit

- P1: 300 liberi
- P2: B (500) + 100 liberi
- P3: 350 liberi
- P4: D (200) + 0 liberi
- P5: C (358) + E (375) + 17 liberi
- P6: A (115) + 10 liberi

Tutti i processi sono stati allocati.

41.5 Worst-Fit: passo per passo

Idea: per ogni processo, scelgo la *più grande* partizione che sia abbastanza grande.

Stato iniziale

$$P_1 = 300,\ P_2 = 600,\ P_3 = 350,\ P_4 = 200,\ P_5 = 750,\ P_6 = 125$$

Processo A = 115 KB

Partizioni che possono contenere 115 KB: tutte.
La più grande è 750 KB (P5).

$$\Rightarrow \text{assegno A a P5}$$

Aggiorno P5:

$$P_5 : 750 - 115 = 635 \text{ KB liberi}$$

Processo B = 500 KB

Partizioni disponibili:

$$P_1 = 300,\ P_2 = 600,\ P_3 = 350,\ P_4 = 200,\ P_5 = 635,\ P_6 = 125$$

Partizioni che possono contenere 500 KB:

$$600,\ 635$$

La più grande è 635 (P5).

$$\Rightarrow \text{assegno B a P5}$$

Aggiorno P5:

$$P_5 : 635 - 500 = 135 \text{ KB liberi}$$

Processo C = 358 KB

Partizioni disponibili:

$$P_1 = 300,\ P_2 = 600,\ P_3 = 350,\ P_4 = 200,\ P_5 = 135,\ P_6 = 125$$

Partizioni che possono contenere 358 KB:

$$600 \text{ (solo P2)}$$

La più grande tra quelle valide è quindi P2 (600).

$$\Rightarrow \text{assegno C a P2}$$

Aggiorno P2:

$$P_2 : 600 - 358 = 242 \text{ KB liberi}$$

Processo D = 200 KB

Partizioni disponibili:

$$P_1 = 300,\ P_2 = 242,\ P_3 = 350,\ P_4 = 200,\ P_5 = 135,\ P_6 = 125$$

Partizioni che possono contenere 200 KB:

$$300,\ 242,\ 350,\ 200$$

La più grande è 350 (P3).

$$\Rightarrow \text{assegno D a P3}$$

Aggiorno P3:

$$P_3 : 350 - 200 = 150 \text{ KB liberi}$$

Processo E = 375 KB

Partizioni disponibili:

$$P_1 = 300,\ P_2 = 242,\ P_3 = 150,\ P_4 = 200,\ P_5 = 135,\ P_6 = 125$$

Controllo se esiste una partizione con dimensione ≥ 375:

$$300 < 375,\ 242 < 375,\ 150 < 375,\ 200 < 375,\ 135 < 375,\ 125 < 375$$

Nessuna partizione è abbastanza grande.

$$\Rightarrow \text{E non può essere allocato}$$

Riepilogo Worst-Fit

- P1: 300 liberi
- P2: C (358) + 242 liberi
- P3: D (200) + 150 liberi
- P4: 200 liberi
- P5: A (115) + B (500) + 135 liberi
- P6: 125 liberi
- E: **non allocato**

41.6 Riepilogo finale Esercizio 2

- **First-fit**: tutti i processi allocati; E entra nella partizione P5 (resto di 392 KB).
- **Best-fit**: tutti i processi allocati; si cerca sempre la partizione valida più piccola.
- **Worst-fit**: il processo da 375 KB non viene allocato, perché le partizioni si sono frammentate in blocchi troppo piccoli.

42 Esercizio 3: Page Table Convenzionale vs Inverted Page Table

42.1 Testo

Si consideri un sistema con:

- spazio di indirizzi logici a 32 bit;
- dimensione di pagina pari a 4 KB;
- memoria fisica massima supportata: 512 MB.

Domanda:

1. Quante entry ci sono in una **page table convenzionale** (single-level)?
2. Quante entry ci sono in una **inverted page table**?

42.2 Passo 1: Interpretare i parametri

- **32-bit logical address space** significa che gli indirizzi logici vanno da 0 a $2^{32} - 1$.
- **Page size = 4 KB** significa che ogni pagina contiene 4 KB di indirizzi.
- **512 MB di memoria fisica** è la dimensione massima della RAM indirizzabile.

In un sistema paginato:

- il numero di **pagine logiche** dipende dallo spazio di indirizzi logici;
- il numero di **frame fisici** dipende dalla dimensione della memoria fisica.

42.3 Passo 2: Calcolare il numero di pagine logiche

Lo spazio logico è:

$$2^{32} \text{ byte}$$

La dimensione di pagina è:

$$4 \text{ KB} = 4 \times 1024 = 4096 \text{ byte} = 2^{12} \text{ byte}$$

Il numero di pagine logiche è:

$$\text{numero di pagine logiche} = \frac{2^{32}}{2^{12}} = 2^{32-12} = 2^{20}$$

- **Perché?** Dividere per 2^{12} equivale a sottrarre 12 all'esponente.
- Quindi abbiamo 2^{20} pagine logiche.

$$\Rightarrow \textbf{numero di pagine logiche} = 2^{20}$$

42.4 Passo 3: Page table convenzionale (single-level)

In una page table convenzionale a un livello:

- c'è una entry per **ogni pagina logica**;
- ogni entry mappa una pagina logica a un frame fisico (o indica che la pagina non è presente).

Quindi il numero di entry è esattamente uguale al numero di pagine logiche:

$$\text{numero di entry nella page table convenzionale} = 2^{20}$$

$$\Rightarrow \boxed{\text{A) Page table convenzionale: } 2^{20} \text{ entry}}$$

42.5 Passo 4: Calcolare il numero di frame fisici

Ora consideriamo la memoria fisica:

$$512 \text{ MB} = 512 \times 1024 \times 1024 \text{ byte}$$

Osserviamo che:

$$512 = 2^9, \quad 1 \text{ MB} = 2^{20} \text{ byte}$$

Quindi:

$$512 \text{ MB} = 2^9 \times 2^{20} = 2^{29} \text{ byte}$$

La dimensione di pagina è sempre:

$$2^{12} \text{ byte}$$

Il numero di frame fisici è:

$$\text{numero di frame fisici} = \frac{2^{29}}{2^{12}} = 2^{29-12} = 2^{17}$$

$$\Rightarrow \textbf{numero di frame fisici} = 2^{17}$$

42.6 Passo 5: Inverted page table

In una **inverted page table**:

- c'è una entry per **ogni frame fisico**, non per ogni pagina logica;
- la dimensione della tabella dipende dalla memoria fisica, non dallo spazio logico.

Quindi il numero di entry è uguale al numero di frame fisici:

$$\text{numero di entry nella inverted page table} = 2^{17}$$

$$\Rightarrow \boxed{\text{B) Inverted page table: } 2^{17} \text{ entry}}$$

42.7 Riepilogo Esercizio 3

- **A) Page table convenzionale (single-level):** 2^{20} entry.
- **B) Inverted page table:** 2^{17} entry.

Osservazione importante:

- la page table convenzionale cresce con lo **spazio logico** (2^{20} pagine);
- la inverted page table cresce con la **memoria fisica** (2^{17} frame);
- questo rende la inverted page table molto più compatta in sistemi con grandi spazi di indirizzamento.

43 Esercizio 4: Bit dell'indirizzo logico e fisico

43.1 Testo

Si consideri uno spazio di indirizzi logici composto da 256 pagine, ciascuna di dimensione 4 KB, mappato su una memoria fisica composta da 64 frame. Determinare:

- quanti bit servono per l'indirizzo logico;
- quanti bit servono per l'indirizzo fisico.

43.2 Passo 1: Analisi dei dati

- Numero di pagine logiche: 256
- Dimensione di ogni pagina: 4 KB = 4096 byte = 2^{12}
- Numero di frame fisici: 64

In un sistema paginato:

$$\text{indirizzo} = \text{numero di pagina/frame} + \text{offset}$$

43.3 Passo 2: Calcolare i bit per il numero di pagina logica

Abbiamo:

$$256 = 2^8$$

- **Perché 8 bit?** Per rappresentare 256 valori distinti (da 0 a 255) servono 8 bit.

$$\Rightarrow \text{numero di pagina logica} = 8 \text{ bit}$$

43.4 Passo 3: Calcolare i bit per l'offset

La dimensione della pagina è:

$$4 \text{ KB} = 4096 \text{ byte} = 2^{12}$$

- **Perché 12 bit?** Per indirizzare 4096 posizioni all'interno della pagina servono 12 bit.

$$\Rightarrow \text{offset} = 12 \text{ bit}$$

43.5 Passo 4: Comporre l'indirizzo logico

$$\text{indirizzo logico} = \text{numero di pagina} + \text{offset}$$

$$\Rightarrow \text{bit indirizzo logico} = 8 + 12 = 20$$

$$\boxed{\text{Indirizzo logico: 20 bit}}$$

43.6 Passo 5: Calcolare i bit per il numero di frame fisico

La memoria fisica ha:

$$64 = 2^6 \text{ frame}$$

- **Perché 6 bit?** Per rappresentare 64 valori distinti servono 6 bit.

$$\Rightarrow \text{numero di frame fisico} = 6 \text{ bit}$$

43.7 Passo 6: Comporre l'indirizzo fisico

L'offset rimane identico (12 bit), perché la dimensione della pagina non cambia.

$$\text{indirizzo fisico} = \text{numero di frame} + \text{offset}$$

$$\Rightarrow \text{bit indirizzo fisico} = 6 + 12 = 18$$

$$\boxed{\text{Indirizzo fisico: 18 bit}}$$

43.8 Riepilogo Esercizio 4

- **Bit indirizzo logico**: 20 bit (8 per la pagina, 12 per l'offset).
- **Bit indirizzo fisico**: 18 bit (6 per il frame, 12 per l'offset).